

Causal Survival - Simulation Study

2026-07-07

Imports

```
library("survival")
library("casebase")
library("sandwich")
library("ggplot2")
library("patchwork")
library("gt")
library("foreach")
library("doParallel")
library("doRNG")
```

Functions

Data Generation

```
## Generate a simulated survival dataset with baseline confounding, a
## treatment effect, and independent random censoring.
generate.data <- function(
  n,
  r,
  l,
  treatment.predictor.coefs,
  intercept,
  outcome.predictor.coefs,
  beta.A,
  censoring.limit,
  constant.HR = TRUE,
  shape = 1
) {

  # Baseline Covariates

  ## Generate 10 baseline covariates from a range of continuous and discrete
  ## distributions. These covariates are subsequently used to generate both
  ## treatment assignment and the event-time distribution.
  X1 <- rnorm(n)
  X2 <- rnorm(n)
  X3 <- runif(n)
  X4 <- runif(n)
  X5 <- rlnorm(n)
  X6 <- rlnorm(n)
  X7 <- rexp(n, rate = r)
  X8 <- rbinom(n, 1, 0.6)
  X9 <- rbinom(n, 1, 0.4)
  X10 <- rpois(n, lambda = 1)

  ## Combine the baseline covariates into a single design matrix for the
  ## treatment and outcome models
  X <- cbind(
    X1, X2, X3, X4, X5,
    X6, X7, X8, X9, X10
  )

  # Input Validation

  ## Ensure that the treatment and outcome coefficient vectors contain one
  ## coefficient for each simulated baseline covariate
  stopifnot(length(treatment.predictor.coefs) == ncol(X))
  stopifnot(length(outcome.predictor.coefs) == ncol(X))

  ## Ensure that all supplied model coefficients are finite
  stopifnot(all(is.finite(treatment.predictor.coefs)))
  stopifnot(all(is.finite(outcome.predictor.coefs)))
}
```

```

# Treatment Assignment

## Generate treatment assignment from a logistic model in which all baseline
## covariates contribute to treatment assignment, thereby inducing
## confounding between treatment and the outcome
weighted.covariates <- as.vector(
  X %*% treatment.predictor.coefs
)

linear.predictor.A <- intercept + weighted.covariates

## Convert the treatment linear predictor to a propensity score and generate
## the binary treatment indicator
A <- rbinom(
  n,
  1,
  plogis(linear.predictor.A)
)

# Event-Time Model and Censoring

## Construct the covariate component of the event-time linear predictor.
## The treatment coefficient beta.A represents the true causal treatment
## effect in the constant-hazard-ratio setting.
weighted.covariates <- as.vector(
  X %*% outcome.predictor.coefs
)

## Generate independent random censoring times from a uniform distribution
## bounded by the specified censoring limit
C <- runif(
  n,
  0,
  censoring.limit
)

## Set the baseline event-time linear predictor. The intercept controls the
## baseline event-time distribution, while the covariates induce prognostic
## heterogeneity across individuals.
linear.predictor.Y <- -1 + weighted.covariates

T.star <- numeric(n)

# Constant Hazard-Ratio Outcome Model

if (constant.HR) {

  ## Add the treatment effect to the event-time linear predictor. Under this
  ## specification, beta.A determines the constant log hazard ratio
  ## associated with treatment.
  linear.predictor.Y <- linear.predictor.Y + (beta.A * A)

  ## Generate event times from a Weibull distribution. The scale parameter

```

```

## depends on the individual-specific linear predictor, allowing both
## treatment and baseline covariates to affect event-time risk.
T.star <- rweibull(
  n,
  shape = shape,
  scale = exp(-linear.predictor.Y / shape)
)

# Time-Varying Hazard-Ratio Outcome Model

} else {

  ## Placeholder for the time-varying treatment-effect specification
  ## (implemented separately when constant.HR = FALSE)
  linear.predictor.Y <- NULL

  T.star <- NULL
}

# Observed Survival Data

## Determine the observed follow-up time as the minimum of the event and
## censoring times
time <- pmin(
  T.star,
  C
)

## Define the event indicator: 1 if the event occurs before or at the
## censoring time, and 0 otherwise
Delta <- as.integer(
  T.star <= C
)

# Return Simulated Dataset

## Combine treatment, observed survival outcomes, and baseline covariates
## into the final simulated analysis dataset
data_set <- data.frame(
  time = as.numeric(time),
  Delta = Delta,
  A = A,
  X
)

return(data_set)
}

```

Modified Case-Base Smooth Hazard Model

```
## Modified implementation of fitSmoothHazard() from the casebase package.
## The function extends the standard case-base fitting procedure to allow
## observation-level weights, including inverse probability weights (IPWs).
FSH <- function(
  formula,
  data,
  time,
  w = rep(1, nrow(data)),
  ratio = 100, ...
) {

  ## Preserve the original function call for reproducibility and method
  ## dispatch in downstream model summaries
  cl <- match.call()

  ## Explicitly expand "." in the model formula so that all covariates in the
  ## supplied dataset are represented in the model specification
  formula <- expand.dot.formula(
    formula,
    data = data
  )

  ## Extract the event variable from the left-hand side of the model formula
  eventVar <- all.vars(formula[[2]])

  ## If the follow-up time variable is not explicitly supplied, infer it from
  ## the data using the event variable and the standard case-base structure
  if (missing(time)) {
    varNames <- checkArgsTimeEvent(
      data = data,
      event = eventVar
    )

    timeVar <- varNames$time
  } else {
    ## Otherwise, use the time variable explicitly supplied by the user
    timeVar <- time
  }

  ## Identify the distinct event types present in the data to determine whether
  ## the analysis involves a single event or competing risks
  typeEvents <- sort(unique(data[[eventVar]]))

  # Case-Base Sampling

  ## Generate the case-base sample when the input data have not already been
  ## sampled. The original dataset is retained so that the supplied observation
  ## weights can be incorporated into the case-base sampling procedure.
  if (!inherits(data, "cbData")) {
```

```

originalData <- as.data.frame(data)
originalData$w <- w

## Sample cases and controls from the person-time data. The sampling ratio
## controls the number of person-moments sampled relative to observed cases.
sampleData <- sampleCaseBase(
  originalData,
  timeVar,
  eventVar,
  comprisk = (length(typeEvents) > 2),
  ratio
)

} else {
  ## If the input is already a case-base dataset, retain it directly.
  ## The original unsampled data are unavailable at this stage.
  originalData <- data.frame()
  sampleData <- data
}

# Weighted Case-Base Model

## Include the case-base sampling offset in the model formula. This offset
## accounts for the sampling mechanism when estimating the hazard function.
formula <- update(formula, ~ . + offset(offset))

## Fit a weighted binomial regression model to the case-base sample.
## Observation-level weights allow the model to incorporate IPWs in addition
## to the case-base sampling structure.
fittingFunction <- function(formula) glm(
  formula,
  data = sampleData,
  family = binomial,
  weights = w
)

## Estimate the smooth hazard model using the weighted case-base sample
out <- fittingFunction(formula)

# Model Object and Metadata

## Preserve the lower-level glm call while replacing the displayed call with
## the original FSH call for consistency with the custom model interface
out$lower_call <- out$call
out$call <- cl

## Store the original data and key variable names for downstream methods
## such as prediction and plotting
out$originalData <- originalData
out$typeEvents <- typeEvents
out$timeVar <- timeVar
out$eventVar <- eventVar

```

```

## Record the number of case and background person-moments in the sampled
## case-base dataset
num_pm <- table(sampleData[[eventVar]])
out$num_cm <- num_pm[2]
out$num_bm <- num_pm[1]
out$formula <- formula

## Store the case-base sampling offset separately, then reset the model's
## offset to zero so that absolute risk estimation can be performed without
## retaining the sampling correction in the prediction stage
out$offset <- out$data$offset
out$data$offset <- 0

## Assign a custom class so that methods defined for single-event case-base
## models can be applied to the fitted object
class(out) <- c("singleEventCB", class(out))

return(out)
}

```


Formula Expansion Helper

```
## Expand "." in a model formula into the complete set of covariates available
## in the supplied dataset. This ensures that the modified FSH function can
## explicitly evaluate formulas containing "." before fitting the model.
expand.dot.formula <- function(formula, data = NULL) {

  ## Check whether the formula contains the "." shorthand for all available
  ## predictors
  if (isTRUE("." %in% all.vars(formula))) {

    ## Preserve the original formula attributes while expanding the terms
    att <- attributes(formula)

    try.terms <- try(
      stats::terms(formula, data = data),
      silent = TRUE
    )

    ## Replace the shorthand formula with its explicitly expanded version when
    ## the terms object can be constructed successfully
    if (!is(try.terms, "try-error")) {
      formula <- formula(try.terms)
    }

    ## Restore the original formula attributes after expansion
    attributes(formula) <- att
  }

  ## Return the expanded formula
  return(formula)
}
```

Restricted Mean Survival Time (RMST)

```
## Numerically integrate a survival curve over time using the trapezoidal rule.  
## The returned vector gives the cumulative RMST evaluated at each time point.  
rmst.fn <- function(time, surv) {  
  dt <- diff(time)  
  mids <- (head(surv, -1) + tail(surv, -1)) / 2  
  c(0, cumsum(mids * dt))  
}
```

Refactor Simulation Results

```
## Convert the list of simulation results into a variable-wise structure.  
## Each element of the returned list contains the corresponding result  
## across all simulation replicates.  
refactor <- function(result) {  
  
  store.by.var <- lapply(  
    names(result[[1]]),  
    function(x) {  
      sapply(result, `[`, x)  
    }  
  )  
  
  names(store.by.var) <- names(result[[1]])  
  
  return(store.by.var)  
}
```

Large-Sample Marginal Effect Estimates

```
## Generate a large simulated dataset and estimate the marginal treatment
## effect using an unadjusted Cox model. The resulting estimates provide
## large-sample benchmarks for evaluating the performance of the estimators
## used in the simulation study.
get.marginal.estimates <- function(
  r, l,
  intercept,
  outcome.predictor.coefs,
  beta.A,
  censoring.limit,
  constant.HR
) {

  # Generate Large Simulated Dataset

  ## Generate a large dataset to approximate the marginal treatment effect.
  ## Setting all treatment-model coefficients to zero removes confounding from
  ## treatment assignment while retaining the covariate effects in the outcome
  ## model.
  dat <- generate.data(
    n = 10000000,
    r = r,
    l = l,
    treatment.predictor.coefs = rep(0, 10),
    intercept = intercept,
    outcome.predictor.coefs = outcome.predictor.coefs,
    beta.A = beta.A,
    censoring.limit = censoring.limit,
    constant.HR = constant.HR
  )

  # Marginal Cox Model

  ## Fit an unadjusted Cox model to estimate the marginal treatment effect.
  ## Under the constant-HR specification, the coefficient of A provides the
  ## large-sample marginal log hazard ratio used as the simulation benchmark.
  ms.cox <- NULL
  psi <- NULL

  if (constant.HR) {

    ms.cox <- coxph(
      Surv(time, Delta) ~ A,
      data = dat
    )

    ## Extract the estimated marginal log hazard ratio for treatment
    psi <- coef(ms.cox)["A"]

    # Time-Varying Treatment Effect
```

```

} else {

  ## Under the time-varying specification, allow the treatment effect to
  ## vary with follow-up time through the A-by-time interaction
  ms.cox <- coxph(
    Surv(time, Delta) ~ A * time,
    data = dat
  )

  ## No single scalar marginal log hazard ratio is extracted when the
  ## treatment effect varies over time
  psi <- NULL
}

# Counterfactual Survival Curves

## Estimate the marginal counterfactual survival curves under no treatment
## and treatment by evaluating the fitted model at A = 0 and A = 1
sf.cox <- survfit(
  ms.cox,
  newdata = data.frame(A = c(0, 1))
)

## Interpolate the counterfactual survival curve under A = 0 onto the common
## simulation time grid
s0.cox <- approx(
  sf.cox$time,
  sf.cox$surv[, 1],
  xout = t.grid,
  method = "constant",
  rule = 2
)$y

## Interpolate the counterfactual survival curve under A = 1 onto the common
## simulation time grid
s1.cox <- approx(
  sf.cox$time,
  sf.cox$surv[, 2],
  xout = t.grid,
  method = "constant",
  rule = 2
)$y

# Marginal RMST Contrast

## Calculate the marginal RMST contrast by integrating the difference between
## the two counterfactual survival curves over the simulation time grid
output <- list(
  psi = psi,
  rmst = rmst.fn(t.grid, s1.cox) - rmst.fn(t.grid, s0.cox)
)

return(output)

```

```
}
```

Intermediate Simulation Step

```
## Repeatedly estimate the large-sample marginal treatment effect and RMST
## contrast across independent simulated datasets. The computation is
## parallelized across multiple worker processes to reduce runtime.
intermediate.step <- function(
  r, l,
  intercept,
  outcome.predictor.coefs,
  beta.A,
  censoring.limit,
  constant.HR
) {

  # Parallel Computing Setup

  ## Create a two-worker parallel cluster and register it for use by foreach
  cl <- parallel::makeCluster(2)
  doParallel::registerDoParallel(cl)

  ## Ensure that the cluster is stopped when the function exits, including if
  ## an error occurs during the simulation
  on.exit(
    parallel::stopCluster(cl),
    add = TRUE
  )

  # Repeated Marginal Effect Estimation

  ## Repeat the marginal-effect calculation over 100 independently generated
  ## datasets. The random-number-generation option ensures reproducible
  ## results across parallel workers.
  result <- foreach(
    i = 1:100,
    .packages = "survival",
    .export = c(
      "generate.data",
      "get.marginal.estimates",
      "t.grid",
      "rmst.fn"
    ),
    .options.RNG = seed
  ) %dornrg% {

    ## Generate the large simulated dataset and obtain the corresponding
    ## marginal log hazard ratio and RMST contrast
    get.marginal.estimates(
      r, l,
      intercept,
      outcome.predictor.coefs,
      beta.A,
      censoring.limit,
      constant.HR
    )
  }
}
```

```
)  
}  
  
# Result Restructuring  
  
## Convert the list of simulation results into the required structured  
## format for subsequent simulation analyses  
return(  
    refactor(result)  
)  
}
```


MS-Cox RMST and Standard Error

```
## Calculate the RMST contrast and its standard error from the marginal
## structural Cox model. The contrast compares restricted mean survival under
## treatment (A = 1) with restricted mean survival under control (A = 0).
cox.rmst.list <- function(
  ms.cox, t.grid, cox.psi.se, survival.curves.data
) {

  ## Estimate the counterfactual survival curves under A = 0 and A = 1
  sf <- survfit(
    ms.cox,
    newdata = survival.curves.data
  )

  ## Interpolate the counterfactual survival curve under A = 0 onto the
  ## specified time grid
  S0 <- approx(
    sf$time,
    sf$surv[, 1],
    t.grid,
    method = "constant",
    rule = 2
  )$y

  ## Interpolate the counterfactual survival curve under A = 1 onto the
  ## specified time grid
  S1 <- approx(
    sf$time,
    sf$surv[, 2],
    t.grid,
    method = "constant",
    rule = 2
  )$y

  # RMST Contrast

  ## Calculate the RMST contrast as the difference between the restricted mean
  ## survival under treatment and control
  cox.rmst <- rmst.fn(t.grid, S1) - rmst.fn(t.grid, S0)

  # Delta-Method Standard Error

  ## Construct the gradient of the RMST functional with respect to the Cox
  ## treatment coefficient. The gradient involves the integral of
  ## S1(t) * log{S1(t)} over the restriction period.
  integrand <- ifelse(
    S1 > 0,
    S1 * log(S1),
    0
  )

  g.cox <- rmst.fn(t.grid, integrand)
```

```
## Propagate uncertainty in the estimated treatment coefficient to the RMST
## contrast using the delta method
cox.rmst.se <- abs(g.cox) * cox.psi.se

return(list(
  cox.rmst = cox.rmst,
  cox.rmst.se = cox.rmst.se
))
}
```

IPW Case-Base RMST and Standard Error

```
## Calculate the RMST contrast and its standard error from the IPW case-base
## model. The RMST standard error is obtained using a numerical delta method.
cb.rmst.list <- function(
  cb.ipw, cb.var, t.grid, constant.HR
) {

  ## Extract the fitted model coefficients and the model terms used to construct
  ## the corresponding design matrix
  beta.hat <- coef(cb.ipw)
  term.form <- delete.response(terms(cb.ipw))

  # Counterfactual Survival Function

  ## Construct the counterfactual survival function under treatment level a
  ## directly from the fitted case-base model
  cb.surv <- function(beta, a, grid) {

    ## Construct prediction data over the specified time grid for treatment
    ## level a
    nd <- data.frame(
      time = grid,
      A = a,
      offset = 0
    )

    ## Generate the model matrix using the fitted model's own terms and align
    ## its columns with the estimated coefficient vector
    X <- model.matrix(term.form, data = nd)
    X <- X[, names(beta), drop = FALSE]

    ## Obtain the estimated hazard function from the model's linear predictor
    lambda <- exp(as.vector(X %*% beta))

    ## Numerically integrate the hazard function using the trapezoidal rule to
    ## obtain the cumulative hazard
    Lambda <- c(
      0,
      cumsum(
        diff(grid) *
        (head(lambda, -1) + tail(lambda, -1)) / 2
      )
    )

    ## Convert the cumulative hazard to the corresponding survival function
    exp(-Lambda)
  }

  # RMST Contrast

  ## Define the RMST contrast as a function of the model coefficient vector.
  ## This function is evaluated at the fitted coefficients and is also used
```

```

## when calculating the numerical gradient.
rmst.diff <- function(beta) {
  rmst.fn(t.grid, cb.surv(beta, 1, t.grid)) -
  rmst.fn(t.grid, cb.surv(beta, 0, t.grid))
}

## Evaluate the RMST contrast at the fitted model coefficients
cb.rmst <- rmst.diff(beta.hat)

# Numerical Gradient

## Calculate the numerical gradient of the RMST contrast with respect to each
## fitted model coefficient using a central finite-difference approximation
p <- length(beta.hat)
m <- length(t.grid)

grad <- matrix(
  NA_real_,
  nrow = m,
  ncol = p
)

colnames(grad) <- names(beta.hat)

## Finite-difference step size
eps <- 1e-6

## Perturb each coefficient in turn to approximate its contribution to the
## RMST contrast
for (j in seq_len(p)) {
  bp <- bm <- beta.hat

  bp[j] <- bp[j] + eps
  bm[j] <- bm[j] - eps

  grad[, j] <- (
    rmst.diff(bp) - rmst.diff(bm)
  ) / (2 * eps)
}

# Delta-Method Standard Error

## Align the covariance matrix with the coefficient ordering used by the
## numerical gradient before applying the delta-method variance formula
cb.var <- cb.var[
  colnames(grad),
  colnames(grad),
  drop = FALSE
]

## Calculate the RMST standard error at each restriction time using the
## delta-method variance  $g' V g$ 
cb.rmst.se <- vapply(

```

```

seq_len(m),
function(i) {
  g <- grad[i, ]

  sqrt(
    drop(
      t(g) %*% cb.var %*% g
    )
  ),
  numeric(1)
)

return(list(
  cb.rmst = cb.rmst,
  cb.rmst.se = cb.rmst.se
))
}

```

Single Simulation Replicate

```
## Run one simulation replicate by generating a dataset, estimating stabilized
## inverse probability weights, fitting the MS-Cox and IPW case-base models,
## and calculating the corresponding treatment effects and RMST contrasts.
run.one.sim <- function(
  n, r, l,
  treatment.predictor.coefs,
  intercept,
  outcome.predictor.coefs,
  beta.A,
  censoring.limit,
  constant.HR,
  i
) {

  # Generate Simulated Dataset

  ## Generate one dataset under the specified data-generating mechanism
  data.set <- generate.data(
    n = n,
    r = r,
    l = l,
    treatment.predictor.coefs = treatment.predictor.coefs,
    intercept = intercept,
    outcome.predictor.coefs = outcome.predictor.coefs,
    beta.A = beta.A,
    censoring.limit = censoring.limit,
    constant.HR = constant.HR
  )

  # Propensity Score Estimation

  ## Estimate the probability of treatment conditional on all baseline
  ## covariates using logistic regression
  p.score <- fitted(
    glm(
      A ~ X1 + X2 + X3 + X4 + X5 +
        X6 + X7 + X8 + X9 + X10,
      data = data.set,
      family = binomial(link = "logit")
    )
  )

  ## Extract the observed treatment indicator and estimate the marginal
  ## probability of receiving treatment
  A <- data.set$A
  p.A1 <- mean(A)

  # Stabilized Inverse Probability Weights

  ## Construct stabilized inverse probability weights using the estimated
  ## propensity scores and the marginal probability of treatment
```

```

ipw.stabilized <- ifelse(
  A == 1,
  p.A1 / p.score,
  (1 - p.A1) / (1 - p.score)
)

# Marginal Structural Cox Model

## Initialize objects used to store the Cox model estimates
ms.cox <- NULL
cox.psi <- NULL
cox.psi.se <- NULL

if (constant.HR) {

  # Constant Hazard Ratio

  ## Fit the marginal structural Cox model using the stabilized IPWs
  ms.cox <- coxph(
    Surv(time, Delta) ~ A,
    data = data.set,
    weights = ipw.stabilized,
    robust = TRUE
  )

  ## Extract the estimated marginal log hazard ratio and its robust standard
  ## error
  cox.psi <- coef(ms.cox)["A"]
  cox.psi.se <- sqrt(diag(vcov(ms.cox))["A"])

} else {

  # Time-Varying Hazard Ratio

  ## Allow the treatment effect to vary over time through the treatment-by-time
  ## interaction
  ms.cox <- coxph(
    Surv(time, Delta) ~ A * time,
    data = data.set,
    weights = ipw.stabilized,
    robust = TRUE
  )

  ## A single scalar treatment-effect estimate is not extracted under the
  ## time-varying specification
  cox.psi <- NULL
  cox.psi.se <- NULL
}

# MS-Cox Counterfactual Survival Curves and RMST

## Define the two treatment profiles used to obtain counterfactual survival

```

```

## curves under A = 0 and A = 1
survival.curves.data <- data.frame(
  A = c(0, 1)
)

## Calculate the RMST contrast and its standard error from the fitted
## marginal structural Cox model
cox.rmst.list <- cox.rmst.list(
  ms.cox = ms.cox,
  t.grid = t.grid,
  cox.psi.se = cox.psi.se,
  survival.curves.data = survival.curves.data
)

## Extract the RMST contrast and its standard error
cox.rmst <- cox.rmst.list$cox.rmst
cox.rmst.se <- cox.rmst.list$cox.rmst.se

# IPW Case-Base Model

## Set the number of controls sampled per case in the case-base design
fsh.ratio <- 100

## Initialize objects used to store the case-base model estimates
cb.ipw <- NULL
cb.psi <- NULL
cb.psi.se <- NULL
cb.var <- NULL

if (constant.HR) {
  # Constant Hazard Ratio

  ## Fit the IPW case-base model using the stabilized inverse probability
  ## weights
  cb.ipw <- FSH(
    Delta ~ A,
    data = data.set,
    ratio = fsh.ratio,
    w = ipw.stabilized
  )

  ## Extract the estimated marginal log hazard ratio and its
  ## heteroskedasticity-consistent standard error
  cb.psi <- coef(cb.ipw)["A"]
  cb.var <- vcovHC(
    cb.ipw,
    type = "HCO"
  )
  cb.psi.se <- sqrt(diag(cb.var)["A"])
} else {

```



```

# Time-Varying Hazard Ratio

## Fit the IPW case-base model with a treatment-by-time interaction to
## allow the treatment effect to vary over follow-up
cb.ipw <- FSH(
  Delta ~ A * pspline(time, df = 4),
  data = data.set,
  ratio = fsh.ratio,
  w = ipw.stabilized
)

## A single scalar treatment-effect estimate and standard error are not
## extracted under the time-varying specification
cb.psi <- NULL
cb.psi.se <- NULL
cb.var <- NULL
}

# IPW Case-Base RMST

## Calculate the RMST contrast and its standard error from the fitted IPW
## case-base model
cb.rmst.list <- cb.rmst.list(
  cb.ipw,
  cb.var,
  t.grid,
  constant.HR
)

## Extract the RMST contrast and its standard error
cb.rmst <- cb.rmst.list$cb.rmst
cb.rmst.se <- cb.rmst.list$cb.rmst.se

# Store Simulation Results

## Return all treatment-effect and RMST estimates from the current simulation
## replicate, together with the replicate identifier
output <- list(
  simulation.number = i,
  cox.psi = cox.psi,
  cox.psi.se = cox.psi.se,
  cb.psi = cb.psi,
  cb.psi.se = cb.psi.se,
  cox.rmst = cox.rmst,
  cox.rmst.se = cox.rmst.se,
  cb.rmst = cb.rmst,
  cb.rmst.se = cb.rmst.se
)

return(output)
}

```

Run a Simulation Study

```
## Run the complete simulation study for a specified data-generating mechanism.
## Each simulation replicate independently generates data, estimates stabilized
## IPWs, fits the MS-Cox and IPW case-base models, and calculates the treatment
## effect and RMST estimates.
simulate <- function(
  n, r, l,
  treatment.predictor.coefs,
  intercept,
  outcome.predictor.coefs,
  beta.A,
  censoring.limit,
  constant.HR
) {

  # Parallel Processing Setup

  ## Allocate approximately 60% of the available CPU cores to the parallel
  ## simulation to balance computational speed with system resource usage
  n.cores <- round(parallel::detectCores() * 0.6)

  cl <- parallel::makeCluster(n.cores)
  doParallel::registerDoParallel(cl)

  ## Ensure that the parallel cluster is stopped when the function exits,
  ## including if an error occurs during the simulation
  on.exit(
    parallel::stopCluster(cl),
    add = TRUE
  )

  # Run Simulation Replicates

  ## Run 1,000 independent simulation replicates in parallel. The RNG option
  ## ensures reproducible random-number generation across parallel workers.
  result <- foreach(
    i = 1:1000,
    .packages = c("survival", "casebase"),
    .export = c(
      "run.one.sim",
      "generate.data",
      "vcovHC", "vcov",
      "FSH", "expand.dot.formula",
      "t.grid", "rmst.fn",
      "cox.rmst.list",
      "cb.rmst.list",
      "shape"
    ),
    .options.RNG = seed
  ) %dorng% {

    ## Execute a single simulation replicate using the specified
```

```

## data-generating mechanism and estimation settings
run.one.sim(
  n,
  r,
  l,
  treatment.predictor.coefs,
  intercept,
  outcome.predictor.coefs,
  beta.A,
  censoring.limit,
  constant.HR,
  i
)
}

# Organize Simulation Results

## Convert the list of replicate-specific results into a compact structure
## suitable for summarizing estimator performance across simulations
return(
  refactor(result)
)
}

```

RMST Plotting Functions

```
## Plot one or more RMST curves on a common set of axes.
## The function automatically selects the plot title, y-axis limits, and legend
## placement based on the supplied curves.
plot.multiple.rmst <- function(
  titles, rmst.list, cols, main.title = ""
) {

  num.types <- length(rmst.list)

  # Plot Configuration

  ## Generate a default title when one is not explicitly supplied. A single
  ## curve uses its corresponding title, while multiple curves are presented
  ## as an RMST comparison.
  if (main.title == "") {
    main.title <- paste(titles[1], "RMST")

    if (num.types > 1) {
      main.title <- "RMST Comparison"
    }
  }

  ## Proceed only when the number of curves, labels, and colors agree and the
  ## number of curves is within the supported plotting range
  if (
    num.types == length(titles) &&
    num.types == length(cols) &&
    num.types > 0 &&
    num.types <= 5
  ) {

    # Determine Plot Limits and Legend Placement

    ## Use the final RMST value to determine an appropriate y-axis range and
    ## position the legend away from the region containing the plotted curves.
    avg.last.point <- mean(
      rmst.list[[1]][-nrow(rmst.list[[1])])
    )

    legend.placement <- "topright"

    if (avg.last.point < -0.3) {
      ylim <- c(
        min(rmst.list[[1]][-nrow(rmst.list[[1])])),
        0
      )

      legend.placement <- "bottomleft"
    } else if (avg.last.point > 0.3) {
      ylim <- c(

```

```

    0,
    max(rmst.list[[1]][-nrow(rmst.list[[1])])
  )
} else {
  ylim <- c(
    min(rmst.list[[1]][-nrow(rmst.list[[1])])),
    max(rmst.list[[1]][-nrow(rmst.list[[1])])
  )
}

## Apply transparency to the plotted curves so that overlapping RMST
## trajectories remain visually distinguishable
alpha.cols <- adjustcolor(
  cols,
  alpha.f = 0.05
)

# Draw RMST Curves

## Initialize the plot using the first RMST curve
matplot(
  t.grid,
  rmst.list[[1]],
  type = "l",
  lty = 1,
  col = alpha.cols[1],
  xlab = "Time",
  ylab = "RMST",
  main = main.title
)

## Overlay any additional RMST curves on the same axes
if (length(rmst.list) > 1) {
  for (i in 2:num.types) {
    matplot(
      t.grid,
      rmst.list[[i]],
      type = "l",
      lty = 1,
      col = alpha.cols[i],
      add = TRUE
    )
  }
}

## Add a horizontal reference line at zero RMST contrast
abline(
  h = 0,
  col = "black",
  lwd = 3,
  lty = 2
)

```

```

# Add Legend

legend(
  legend.placement,
  legend = titles,
  col = cols[1:num.types],
  lwd = 2,
  lty = 1
)

} else {

  ## Report an error when the supplied plotting arguments are inconsistent
  cat("error")

  if (length(titles) != num.types) {
    cat("mismatch")
  }
}
}

```

Generate RMST Plots

```
## Generate the complete set of RMST figures used to compare the estimated
## treatment effects from MS-Cox and case-base models against the marginal
## (true) RMST curve.
##
## When save.plots = TRUE, each figure is written to a separate PDF file.
plot.all.rmst <- function(
  true.rmst, cox.rmst, cb.rmst, save.plots = FALSE
) {

  # Marginal RMST

  ## Plot the marginal RMST curve alone to display the target causal estimand
  if (save.plots) {
    pdf(
      "true-rmst.pdf",
      width = 8,
      height = 5
    )
  }

  plot.multiple.rmst(
    titles = c("Marginal"),
    rmst.list = list(true.rmst),
    cols = "purple"
  )

  if (save.plots) {
    dev.off()
  }

  # MS-Cox vs. Marginal RMST

  ## Compare the MS-Cox RMST estimate with the marginal RMST target
  if (save.plots) {
    pdf(
      "cox-rmst.pdf",
      width = 8,
      height = 5
    )
  }

  plot.multiple.rmst(
    titles = c(
      "MS-Cox",
      "Marginal"
    ),
    rmst.list = list(
      cox.rmst,
      true.rmst
    ),
    cols = c(
```

```

    "red",
    "black"
  ),
  main.title = "MS-Cox RMST"
)

if (save.plots) {
  dev.off()
}

# Case-Base vs. Marginal RMST

## Compare the IPW case-base RMST estimate with the marginal RMST target
if (save.plots) {
  pdf(
    "cb-rmst.pdf",
    width = 8,
    height = 5
  )
}

plot.multiple.rmst(
  titles = c(
    "Case Base",
    "Marginal"
  ),
  rmst.list = list(
    cb.rmst,
    true.rmst
  ),
  cols = c(
    "blue",
    "black"
  ),
  main.title = "Case Base RMST"
)

if (save.plots) {
  dev.off()
}

# Overall RMST Comparison

## Compare both estimators simultaneously against the marginal RMST target
if (save.plots) {
  pdf(
    "all-rmst.pdf",
    width = 8,
    height = 5
  )
}

plot.multiple.rmst(

```



```
titles = c(
  "MS-Cox",
  "Case Base",
  "Marginal"
),
rmst.list = list(
  cox.rmst,
  cb.rmst,
  true.rmst
),
cols = c(
  "red",
  "blue",
  "black"
)
)

if (save.plots) {
  dev.off()
}
}
```

RMST Simulation Performance Summary

```
## Summarize the finite-sample performance of an RMST estimator across
## simulation replicates. The function calculates the Monte Carlo mean,
## empirical standard deviation, mean estimated standard error, bias, RMSE,
## and confidence-interval coverage at the specified restriction times.
##
## The marginal RMST values are treated as the target estimand against which
## the simulation estimates are evaluated.
get.rmst.summary <- function(
  tau.points,
  rmst.matrix,
  rmst.se.matrix,
  marginal.rmst.means,
  alpha = 0.05
) {

  ## Number of simulation replicates and standard-normal critical value for
  ## the nominal (1 - alpha) confidence intervals
  n <- ncol(rmst.matrix)
  z <- qnorm(1 - alpha / 2)

  # Point Estimation and Variability

  ## Monte Carlo mean of the estimated RMST contrast across simulation
  ## replicates at each selected restriction time
  rmst.means <- rowMeans(
    rmst.matrix[tau.points, ]
  )

  ## Empirical standard deviation of the RMST estimates across simulation
  ## replicates, representing the observed Monte Carlo variability
  rmst.sd <- apply(
    rmst.matrix[tau.points, ],
    1,
    sd
  )

  ## Average model-based standard error reported by the estimator across
  ## simulation replicates
  rmst.se.mean <- rowMeans(
    rmst.se.matrix[tau.points, ]
  )

  # Bias and RMSE

  ## Monte Carlo bias relative to the marginal RMST target
  rmst.biases <- rmst.means - marginal.rmst.means

  ## Monte Carlo standard error of the estimated bias
  rmst.bias.mcse <- rmst.sd / sqrt(n)

  ## Root mean squared error relative to the marginal RMST target, combining
```

```

## both systematic bias and sampling variability
rmst.rmse <- sqrt(
  rowMeans(
    (
      rmst.matrix[tau.points, ] -
      marginal.rmst.means
    )^2
  )
)

# Confidence-Interval Coverage

## Construct nominal (1 - alpha) Wald confidence intervals for the RMST
## contrast using the estimated standard errors from each simulation
lower <- rmst.matrix[tau.points, ] -
  z * rmst.se.matrix[tau.points, ]

upper <- rmst.matrix[tau.points, ] +
  z * rmst.se.matrix[tau.points, ]

## Estimate empirical coverage as the proportion of simulation replicates
## whose confidence interval contains the marginal RMST target
coverage <- rowMeans(
  lower <= marginal.rmst.means &
  upper >= marginal.rmst.means
)

## Monte Carlo standard error of the estimated coverage probability, treating
## coverage as a binomial proportion across simulation replicates
coverage.mcse <- sqrt(
  coverage * (1 - coverage) / n
)

# Assemble Summary Table

## Initialize the summary matrix with one row per restriction time and one
## column for each simulation performance metric
k <- length(tau.points)

rmst.summary <- matrix(
  NA_real_,
  nrow = k,
  ncol = 8
)

colnames(rmst.summary) <- c(
  "mean",
  "sd",
  "se.mean",
  "bias",
  "bias.mcse",
  "rmse",
  "coverage",

```

```

    "coverage.mcse"
  )

  ## Populate the summary matrix with the calculated performance measures
  rmst.summary[, "mean"] <- rmst.means
  rmst.summary[, "sd"] <- rmst.sd
  rmst.summary[, "se.mean"] <- rmst.se.mean
  rmst.summary[, "bias"] <- rmst.biases
  rmst.summary[, "bias.mcse"] <- rmst.bias.mcse
  rmst.summary[, "rmse"] <- rmst.rmse
  rmst.summary[, "coverage"] <- coverage
  rmst.summary[, "coverage.mcse"] <- coverage.mcse

  return(rmst.summary)
}

```

Print RMST Simulation Summary

```
## Print the simulation performance summary for an RMST estimator in a
## human-readable format. Results are displayed separately for each selected
## restriction time, including the Monte Carlo mean, bias, RMSE, estimated
## standard error, empirical standard deviation, and confidence-interval
## coverage.
print.rmst.summary <- function(
  rmst.summary,
  tau.values,
  model
) {

  # Print Model Header

  ## Display the name of the estimator being summarized
  cat("\n===== \n")
  cat(model, "\n")
  cat("===== \n")

  cat("\nRestricted Mean Survival Time Summary\n")
  cat("----- \n")

  # Print Results at Each Restriction Time

  ## Display the simulation performance metrics separately at each selected
  ## restriction time
  for (i in seq_along(tau.values)) {

    cat("\nTau =", tau.values[i], "\n")
    cat("----- \n")

    ## Monte Carlo mean of the estimated RMST contrast
    cat(
      "Mean RMST:",
      rmst.summary[i, "mean"],
      "\n"
    )

    ## Monte Carlo bias and its Monte Carlo standard error
    cat(
      "Bias:",
      rmst.summary[i, "bias"],
      " | MC-SE:",
      rmst.summary[i, "bias.mcse"],
      "\n"
    )

    ## Root mean squared error of the RMST estimator
    cat(
      "RMSE:",
      rmst.summary[i, "rmse"],
      "\n"
    )
  }
}
```

```

)

## Compare the average model-based standard error with the empirical
## standard deviation across simulation replicates
cat(
  "Mean Estimated SE:",
  rmst.summary[i, "se.mean"],
  " | Empirical SD:",
  rmst.summary[i, "sd"],
  "\n"
)

## Empirical confidence-interval coverage and its Monte Carlo standard error
cat(
  "Coverage:",
  rmst.summary[i, "coverage"],
  " | MC-SE:",
  rmst.summary[i, "coverage.mcse"],
  "\n"
)
}

cat("\n")
}

```

Print True Marginal RMST

```
## Print the true marginal RMST contrast at each selected restriction time.
## The true value is obtained by averaging the simulated marginal RMST values
## across the corresponding time point.
print.true.rmst <- function(
  true.rmst,
  tau.points,
  tau.values
) {

  # Print Header

  ## Identify the values being reported as the true marginal RMST
  cat("\n=====\\n")
  cat("\nTrue Mean Restricted Mean Survival Time\\n")
  cat("=====\\n")

  # Print True RMST at Each Restriction Time

  ## Display the true marginal RMST corresponding to each selected restriction
  ## time
  for (i in seq_along(tau.values)) {
    cat(
      tau.values[i],
      ":",
      mean(true.rmst[tau.points[i], ]),
      "\\n"
    )
  }

  cat("\\n")
}
```

Plot Hazard Ratio Distributions

```
## Plot the sampling distributions of the estimated hazard ratios from the
## MS-Cox and Case-Base estimators and compare them with the true marginal HR.
plot.hr <- function(
  true.hr,
  cox.hr,
  cb.hr,
  save.plot = FALSE
) {

  # Set Up Optional PDF Output

  ## Open a PDF device when the plot is requested to be saved
  if (save.plot) {
    pdf("hr.pdf", width = 8, height = 5)
  }

  # Plot Estimated HR Distributions

  ## Plot the sampling distribution of the MS-Cox HR estimates
  hist(
    cox.hr,
    breaks = 50,
    col = rgb(1, 0, 0, 0.5),
    border = "red",
    xlab = "Hazard Ratio",
    main = "Hazard Ratio Comparison"
  )

  ## Overlay the sampling distribution of the Case-Base HR estimates
  hist(
    cb.hr,
    breaks = 50,
    col = rgb(0, 0, 1, 0.5),
    border = "blue",
    add = TRUE
  )

  # Add True HR Reference

  ## Mark the true marginal HR with a vertical reference line
  abline(
    v = true.hr,
    col = "black",
    lwd = 3,
    lty = 2
  )

  # Add Legend

  ## Identify the two estimator distributions and the true marginal HR
  legend(
```



```

"topright",
legend = c(
  "MS-Cox HR",
  "Case Base HR",
  "True HR"
),
fill = c(
  rgb(1, 0, 0, 0.5),
  rgb(0, 0, 1, 0.5),
  NA
),
border = c(
  "red",
  "blue",
  NA
),
lty = c(
  NA,
  NA,
  2
),
lwd = c(
  NA,
  NA,
  3
),
col = c(
  NA,
  NA,
  "black"
)
)

# Close Optional PDF Output

## Close the PDF device after the plot has been written
if (save.plot) {
  dev.off()
}
}

```

Plot Hazard Ratio Estimation Errors

```
## Plot the sampling distributions of the estimation errors for the MS-Cox and
## Case-Base HR estimators. An error of zero indicates an estimate equal to the
## true marginal HR.
plot.hr.errors <- function(
  true.hr,
  cox.hr,
  cb.hr,
  save.plot = FALSE
) {

  # Calculate HR Estimation Errors

  ## Calculate the deviation of each estimated HR from the true marginal HR
  cox.hr.error <- cox.hr - true.hr
  cb.hr.error <- cb.hr - true.hr

  # Set Up Optional PDF Output

  ## Open a PDF device when the plot is requested to be saved
  if (save.plot) {
    pdf("hr-errors.pdf", width = 8, height = 5)
  }

  # Plot HR Error Distributions

  ## Plot the sampling distribution of the MS-Cox HR estimation errors
  hist(
    cox.hr.error,
    breaks = 50,
    col = rgb(1, 0, 0, 0.5),
    border = "red",
    xlab = "HR Errors",
    main = "HR Errors Comparison"
  )

  ## Overlay the sampling distribution of the Case-Base HR estimation errors
  hist(
    cb.hr.error,
    breaks = 50,
    col = rgb(0, 0, 1, 0.5),
    border = "blue",
    add = TRUE
  )

  # Add Zero-Error Reference

  ## Mark zero error, corresponding to an estimate exactly equal to the true
  ## marginal HR
  abline(
    v = 0,
    col = "black",
```

```

    lwd = 3,
    lty = 2
)

# Add Legend

## Identify the estimation-error distributions for the two estimators
legend(
  "topleft",
  legend = c(
    "MS-Cox HR Error",
    "Case Base HR Error"
  ),
  fill = c(
    rgb(1, 0, 0, 0.5),
    rgb(0, 0, 1, 0.5)
  ),
  border = c(
    "red",
    "blue"
  )
)

# Close Optional PDF Output

## Close the PDF device after the plot has been written
if (save.plot) {
  dev.off()
}
}

```

Summarize Hazard Ratio Simulation Performance

```
## Calculate simulation performance measures for a hazard ratio estimator,
## including bias, RMSE, average estimated standard error, confidence-interval
## coverage, and either Type I error or power.
get.hr.summary <- function(
  true.hr,
  psi.vector,
  se.vector,
  alpha = 0.05,
  power = TRUE
) {

  # Set Up Simulation Quantities

  ## Number of simulation replicates and normal critical value for the
  ## confidence intervals
  n <- length(psi.vector)
  z <- qnorm(1 - alpha / 2)

  ## Transform log-hazard-ratio estimates to the hazard-ratio scale
  hr.vector <- exp(psi.vector)

  # Point-Estimation Performance

  ## Monte Carlo mean, empirical standard deviation, and bias of the estimated
  ## hazard ratios
  hr.mean <- exp(mean(psi.vector))
  hr.emp.sd <- sd(hr.vector)
  hr.bias <- hr.mean - true.hr

  ## Monte Carlo standard error of the estimated bias
  hr.bias.mcse <- hr.emp.sd / sqrt(n)

  ## Mean squared error and root mean squared error relative to the true HR
  hr.mse <- mean((hr.vector - true.hr)^2)
  hr.rmse <- sqrt(hr.mse)

  # Standard Error and Confidence-Interval Performance

  ## Average model-based standard error across simulation replicates
  hr.se.mean <- mean(se.vector)

  ## Construct Wald confidence intervals on the HR scale
  hr.ci.lower <- exp(psi.vector - z * se.vector)
  hr.ci.upper <- exp(psi.vector + z * se.vector)

  # Type I Error / Power and Coverage

  ## Calculate the proportion of confidence intervals excluding the null HR of
  ## one. This corresponds to Type I error when the true HR equals one and to
  ## empirical power when the true HR differs from one.
  t1.error.or.power <- mean(
```

```

    hr.ci.lower > 1 | hr.ci.upper < 1
  )

  ## Calculate empirical coverage of the confidence intervals for the true HR
  hr.coverage <- mean(
    hr.ci.lower <= true.hr &
    true.hr <= hr.ci.upper
  )

  ## Monte Carlo standard error of the empirical coverage probability
  hr.coverage.mcse <- sqrt(
    hr.coverage * (1 - hr.coverage) / n
  )

  # Store Summary Statistics and Confidence Intervals

  ## Store the simulation performance measures and individual confidence
  ## intervals in a list for downstream analysis and reporting
  results <- list(
    summary = c(
      mean          = hr.mean,
      emp.sd        = hr.emp.sd,
      bias          = hr.bias,
      bias.mcse     = hr.bias.mcse,
      mse           = hr.mse,
      rmse          = hr.rmse,
      se.mean       = hr.se.mean,
      t1.error.or.power = t1.error.or.power,
      coverage      = hr.coverage,
      coverage.mcse = hr.coverage.mcse
    ),
    ci = cbind(
      lower = hr.ci.lower,
      upper = hr.ci.upper
    )
  )

  return(results)
}

```

Print Hazard Ratio Simulation Summary

```

## Print the main simulation performance measures for a hazard ratio estimator
## in a human-readable format.
print.hr.summary <- function(
  hr.stats.summary,
  model
) {

  # Print Model Header

```

```

## Identify the estimator corresponding to the reported results
cat("\n=====\\n")
cat(model, "\\n")
cat("=====\\n")

cat("\\nHazard Ratio Summary\\n")
cat("-----\\n")

# Print Performance Measures

## Monte Carlo mean and bias of the estimated hazard ratio
cat(
  "Mean HR:",
  hr.stats.summary["mean"],
  "\\n"
)

## Bias and its Monte Carlo standard error
cat(
  "Bias:",
  hr.stats.summary["bias"],
  " | MC-SE:",
  hr.stats.summary["bias.mcse"],
  "\\n"
)

## Mean squared error and root mean squared error
cat(
  "MSE:",
  hr.stats.summary["mse"],
  "\\n"
)

cat(
  "RMSE:",
  hr.stats.summary["rmse"],
  "\\n"
)

## Compare the average estimated standard error with the empirical standard
## deviation across simulation replicates
cat(
  "Mean Estimated SE:",
  hr.stats.summary["se.mean"],
  " | Empirical SD:",
  hr.stats.summary["emp.sd"],
  "\\n"
)

## Report the proportion of confidence intervals excluding the null HR of one
cat(
  "Type I Error/Power:",
  hr.stats.summary["t1.error.or.power"],

```

```

    "\n"
)

## Report empirical confidence-interval coverage and its Monte Carlo standard
## error
cat(
  "Coverage:",
  hr.stats.summary["coverage"],
  " | MC-SE:",
  hr.stats.summary["coverage.mcse"],
  "\n"
)

cat("\n")
}

```

Print True Hazard Ratio

```
## Print the true marginal hazard ratio used as the reference value in the
## simulation study.
print.true.hr <- function(true.hr) {

  cat("\n===== \n")
  cat("True HR:", true.hr, "\n")
  cat("===== \n")
}
```


Calculate and Print HR Simulation Summaries

```
## Calculate and print simulation performance summaries for both the MS-Cox and
## Case-Base estimators using their estimated log-HR coefficients and standard
## errors.
summarize.print.hr <- function(
  true.hr,
  cox.psi, cox.psi.se,
  cb.psi, cb.psi.se
) {

  # Calculate Estimator-Specific Summaries

  ## Summarize the simulation performance of the MS-Cox hazard ratio estimator
  cox.psi.stats <- get.hr.summary(
    true.hr,
    cox.psi,
    cox.psi.se
  )

  ## Summarize the simulation performance of the Case-Base hazard ratio
  ## estimator
  cb.psi.stats <- get.hr.summary(
    true.hr,
    cb.psi,
    cb.psi.se
  )

  # Print Estimator Results

  ## Print the MS-Cox simulation summary
  print.hr.summary(
    cox.psi.stats$summary,
    "MS-Cox"
  )

  ## Print the Case-Base simulation summary
  print.hr.summary(
    cb.psi.stats$summary,
    "Case Base"
  )

  # Return Summary Objects

  ## Return both estimator-specific results for downstream analysis
  return(list(
    cox.psi.stats,
    cb.psi.stats
  ))
}
```

Plot Hazard Ratio Confidence Intervals

```
## Plot a random sample of simulation-specific hazard ratio estimates and their
## corresponding confidence intervals. The true HR and null value are included
## as reference lines to visually assess estimator accuracy and interval
## coverage.
plot.hr.ci <- function(
  true.value,
  title,
  ci.list,
  hr.list,
  colour = "red",
  coverage = 0,
  seed = 123
) {

  # Select Simulation Replicates

  ## Set the seed to ensure that the same simulation replicates are selected
  ## each time the plot is generated
  set.seed(seed)
  x = 1:100
  sampled.indices <- sample(1:length(hr.list), 100)

  # Construct Confidence-Interval Plot

  ## Plot the confidence intervals as shaded bands, with the corresponding
  ## hazard ratio estimates overlaid as individual simulation trajectories
  plot <- ggplot() +
    geom_ribbon(
      aes(
        x = x,
        ymin = ci.list[[1]][sampled.indices],
        ymax = ci.list[[2]][sampled.indices]
      ),
      fill = colour,
      alpha = 0.5,
      show.legend = FALSE
    ) +
    geom_line(
      aes(
        x = x,
        y = hr.list[sampled.indices],
        colour = title
      )
    ) +

  ## Add the true marginal HR as a horizontal reference line
  geom_hline(
    aes(
      yintercept = true.value,
      colour = "True HR",
      linetype = "True HR"
    )
  )
}
```

```

    )
  ) +

  ## Add the null HR of one as a reference for hypothesis testing
  geom_hline(
    aes(
      yintercept = 1,
      colour = "Null value",
      linetype = "Null value"
    )
  ) +

  # Configure Plot Appearance

  ## Assign distinct colours to the estimator, true HR, and null value
  scale_colour_manual(
    name = "",
    values = c(
      setNames(colour, title),
      "True HR" = "black",
      "Null value" = "purple"
    )
  ) +

  ## Use distinct line types to differentiate the estimator from the
  ## reference values
  scale_linetype_manual(
    name = "",
    values = c(
      title = "solid",
      "True HR" = "dashed",
      "Null value" = "dotted"
    )
  ) +

  ## Label the axes and include the empirical coverage in the plot title
  labs(
    x = "Simulation",
    y = "Hazard ratio",
    title = paste(
      title,
      "HR Confidence Intervals - Coverage:",
      coverage
    )
  ) +

  theme_bw()

# Return Plot

## Return the ggplot object so that it can be displayed, modified, or combined
## with other plots
return(plot)

```

}

Plot HR Confidence Intervals for Both Estimators

```
## Generate and combine confidence-interval plots for the MS-Cox and Case-Base
## estimators, allowing direct visual comparison of their interval coverage and
## estimated hazard ratios across simulation replicates.
plot.all.hr.ci <- function(
  true.hr,
  cox.hr, cox.hr.stats,
  cb.hr, cb.hr.stats,
  seed = 123,
  save.plot = FALSE
) {

  # Generate MS-Cox Confidence-Interval Plot

  ## Extract the MS-Cox confidence intervals and estimated HRs for plotting
  cox.hr.ci.plot <- plot.hr.ci(
    true.hr,
    title = "MS-Cox",
    list(
      cox.hr.stats$ci[, "lower"],
      cox.hr.stats$ci[, "upper"]
    ),
    cox.hr,
    colour = "red",
    coverage = cox.hr.stats$summary["coverage"],
    seed = seed
  )

  # Generate Case-Base Confidence-Interval Plot

  ## Extract the Case-Base confidence intervals and estimated HRs for plotting
  cb.hr.ci.plot <- plot.hr.ci(
    true.hr,
    title = "Case Base",
    list(
      cb.hr.stats$ci[, "lower"],
      cb.hr.stats$ci[, "upper"]
    ),
    cb.hr,
    colour = "blue",
    coverage = cb.hr.stats$summary["coverage"],
    seed = seed
  )

  # Combine Estimator-Specific Plots

  ## Stack the two estimator-specific plots vertically and combine their legends
  combined_plot <- (cox.hr.ci.plot / cb.hr.ci.plot) +
    plot_layout(guides = "collect")

  # Optionally Save Combined Plot
```

```
## Save the combined figure as a PDF when requested
if (save.plot) {
  ggsave(
    "hr-ci.pdf",
    combined_plot,
    width = 8,
    height = 8
  )
}
```

Run a Complete Simulation Study

```
## Run the complete simulation study for a specified sample size, treatment
## effect, and censoring level. The function evaluates both RMST and hazard
## ratio performance for the MS-Cox and Case-Base estimators, with optional
## generation and saving of diagnostic plots.
run.complete.ss <- function(
  n,
  beta.A,
  censoring.limit,
  marginal.estimates,
  generate.plots = FALSE,
  save.plots = FALSE
) {

  # Determine Whether Plots Are Required

  ## Generate plots when either display or file output has been requested
  need.plots <- generate.plots || save.plots

  # Run Simulation Replicates

  ## Generate the simulation results for the specified data-generating scenario
  ss.out <- simulate(
    n, r, l,
    treatment.predictor.coefs,
    intercept,
    outcome.predictor.coefs,
    beta.A,
    censoring.limit,
    constant.HR
  )

  # Restricted Mean Survival Time (RMST)

  # Plot RMST Results

  ## Generate RMST plots comparing the two estimators with the true marginal
  ## RMST curve when plotting has been requested
  if (need.plots) {
    plot.all.rmst(
      marginal.estimates$rmst,
      ss.out$cox.rmst,
      ss.out$cb.rmst,
      save.plots = save.plots
    )
  }

  # Select Restriction Times for Summary

  ## Identify integer restriction times greater than zero for tabular summaries
  tau.points <- which(
```

```

    (t.grid != 0) &
      (t.grid == floor(t.grid))
  )

  # Calculate True Marginal RMST Values

  ## Calculate the Monte Carlo mean and standard deviation of the marginal RMST
  ## values used as the true reference values
  marginal.rmst.means <- rowMeans(
    marginal.estimates$rmst[tau.points, ]
  )

  marginal.rmst.sd <- apply(
    marginal.estimates$rmst[tau.points, ],
    1,
    sd
  )

  # Summarize RMST Simulation Performance

  ## Calculate simulation performance measures for the MS-Cox RMST estimator
  cox.rmst.summary <- get.rmst.summary(
    tau.points,
    ss.out$cox.rmst,
    ss.out$cox.rmst.se,
    marginal.rmst.means
  )

  ## Calculate simulation performance measures for the Case-Base RMST estimator
  cb.rmst.summary <- get.rmst.summary(
    tau.points,
    ss.out$cb.rmst,
    ss.out$cb.rmst.se,
    marginal.rmst.means
  )

  ## Define the restriction times reported in the simulation summary
  tau.values <- seq(1, 4, by = 1)

  # Print RMST Results

  ## Print the true marginal RMST values at the selected restriction times
  print.true.rmst(
    marginal.estimates$rmst,
    tau.points,
    tau.values
  )

  ## Print the MS-Cox RMST simulation performance summary
  print.rmst.summary(
    cox.rmst.summary,
    tau.values,
    "MS-Cox RMST"
  )

```



```

)

## Print the Case-Base RMST simulation performance summary
print.rmst.summary(
  cb.rmst.summary,
  tau.values,
  "Case Base RMST"
)

# Hazard Ratio (HR)

# Determine the True Marginal HR

## Calculate the true marginal hazard ratio by averaging the simulated
## marginal log-HR values and transforming to the HR scale
true.hr <- exp(mean(marginal.estimates$psi))

# Plot HR Results

if (need.plots) {

  ## Plot the distribution of the marginal HR values used to determine the
  ## simulation reference value
  if (save.plots) {
    pdf("true-hr.pdf", width = 8, height = 5)
  }

  hist(
    exp(marginal.estimates$psi),
    breaks = 30,
    xlab = "HR",
    main = "Marginal HR"
  )

  if (save.plots) {
    dev.off()
  }

  ## Compare the sampling distributions of the MS-Cox and Case-Base HR
  ## estimates against the true marginal HR
  plot.hr(
    true.hr,
    exp(ss.out$cox.psi),
    exp(ss.out$cb.psi),
    save.plot = save.plots
  )

  ## Compare the estimation errors of the two HR estimators relative to the
  ## true marginal HR
  plot.hr.errors(
    true.hr,
    exp(ss.out$cox.psi),

```

```

    exp(ss.out$cb.psi),
    save.plot = save.plots
  )
}

# Print True HR

## Report the true marginal HR used as the reference value for evaluating
## estimator performance
print.true.hr(true.hr)

# Summarize HR Simulation Performance

## Calculate and print simulation performance measures for both HR estimators
all.hr.stats <- summarize.print.hr(
  true.hr,
  ss.out$cox.psi,
  ss.out$cox.psi.se,
  ss.out$cb.psi,
  ss.out$cb.psi.se
)

# Plot HR Confidence Intervals

## Visualize confidence intervals from both estimators across simulation
## replicates, including the true HR and null value
if (need.plots) {
  plot.all.hr.ci(
    true.hr,
    exp(ss.out$cox.psi),
    all.hr.stats[[1]],
    exp(ss.out$cb.psi),
    all.hr.stats[[2]],
    seed = seed,
    save.plot = save.plots
  )
}
}

```

Simulation Study

Data Generation Parameters

```
seed <- 123

tau.max <- 4
t.grid <- seq(0, tau.max, by = 0.1)

r = 2
l = 1.5

treatment.predictor.coefs <- c(
  0.7, -0.4, 0.5, 0.2, 0.6,
  -0.3, 0.1, -0.2, 0.4, -0.5
)

outcome.predictor.coefs <- c(
  0.6, -0.3, 0.4, 0.1, 0.5,
  -0.2, 0.05, -0.1, 0.3, -0.4
)

intercept <- 0.3
constant.HR <- TRUE
shape = 1
```

Setting 1: $\beta.A = 0$, $n = 1000$ (Null)

```
n1 = 1000
beta.A1 <- 0
censoring.limit.1 <- 5
```

Setting 2: $\beta.A = 1$, $n = 1000$ (Strong)

```
n2 = 1000
beta.A2 <- 1
censoring.limit.2 <- 5
```

Setting 3: $\beta.A = 0.5$, $n = 1000$ (Moderate)

```
n3 = 1000
beta.A3 <- 0.5
censoring.limit.3 <- 5
```

Setting 4: $\beta.A = 0.5$, $n = 500$ (Moderate)

```
n4 = 500
beta.A4 <- 0.5
censoring.limit.4 <- 5
```

Setting 5: $\beta.A = 0.5$, $n = 250$ (Moderate)

```
n5 = 250
beta.A5 <- 0.5
censoring.limit.5 <- 5
```

Setting 6: $\beta.A = 0.5$, $n = 1000$ (Heavy Censoring)

```
n6 = 1000
beta.A6 <- 0.5
censoring.limit.6 <- 3
```

Extract the marginal estimates for all settings

Moderate Beta

```
true.estimates.moderate <- intermediate.step(  
  r, 1,  
  intercept,  
  outcome.predictor.coefs,  
  beta.A3,  
  censoring.limit.1,  
  constant.HR  
)
```

Null Beta

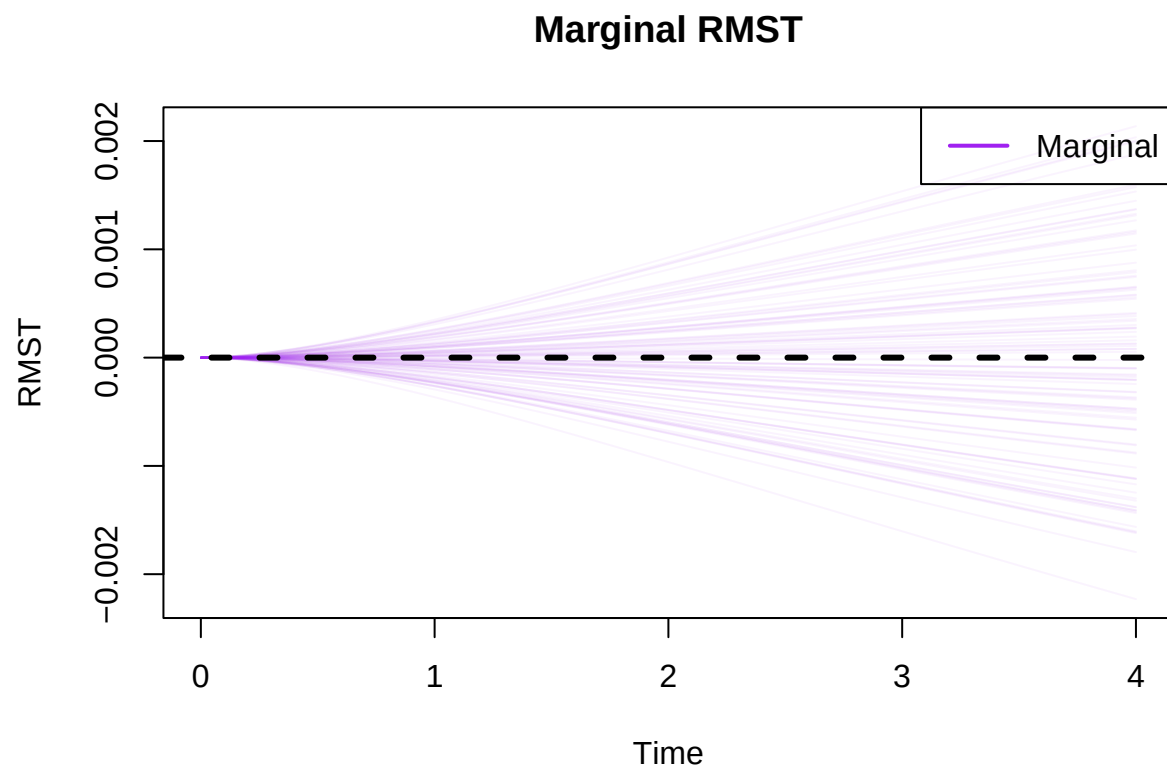
```
true.estimates.null <- intermediate.step(  
  r, 1,  
  intercept,  
  outcome.predictor.coefs,  
  beta.A1,  
  censoring.limit.1,  
  constant.HR  
)
```

Strong Beta

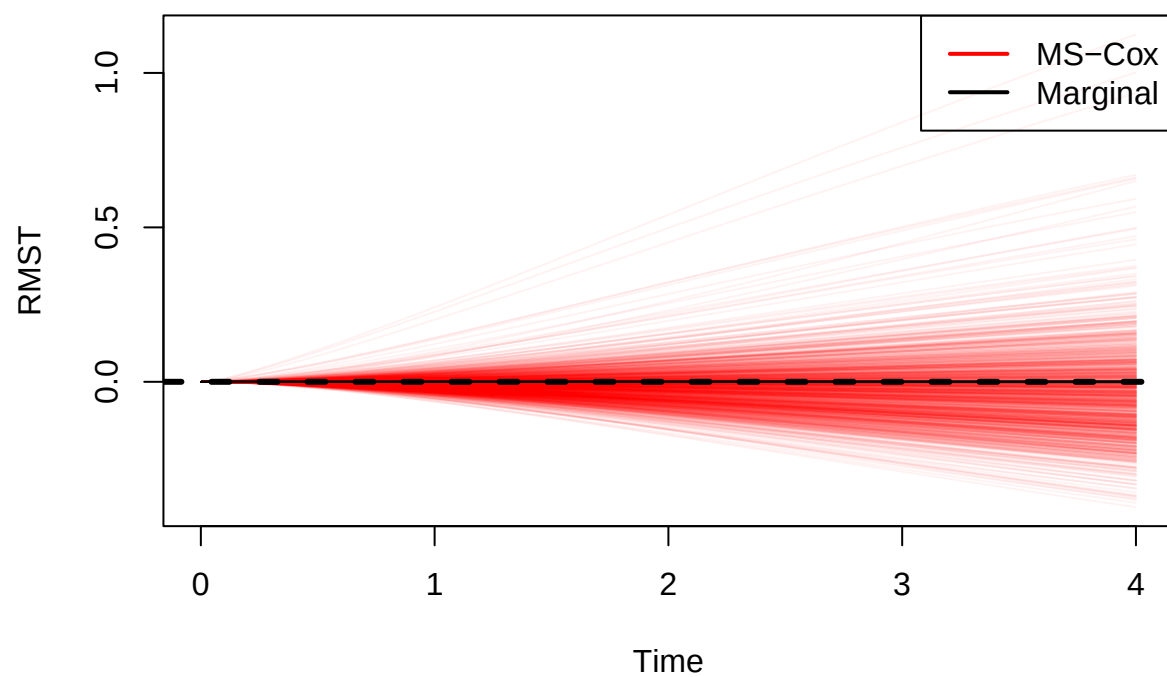
```
true.estimates.strong <- intermediate.step(  
  r, 1,  
  intercept,  
  outcome.predictor.coefs,  
  beta.A2,  
  censoring.limit.1,  
  constant.HR  
)
```

Run Simulation Study: Setting 1

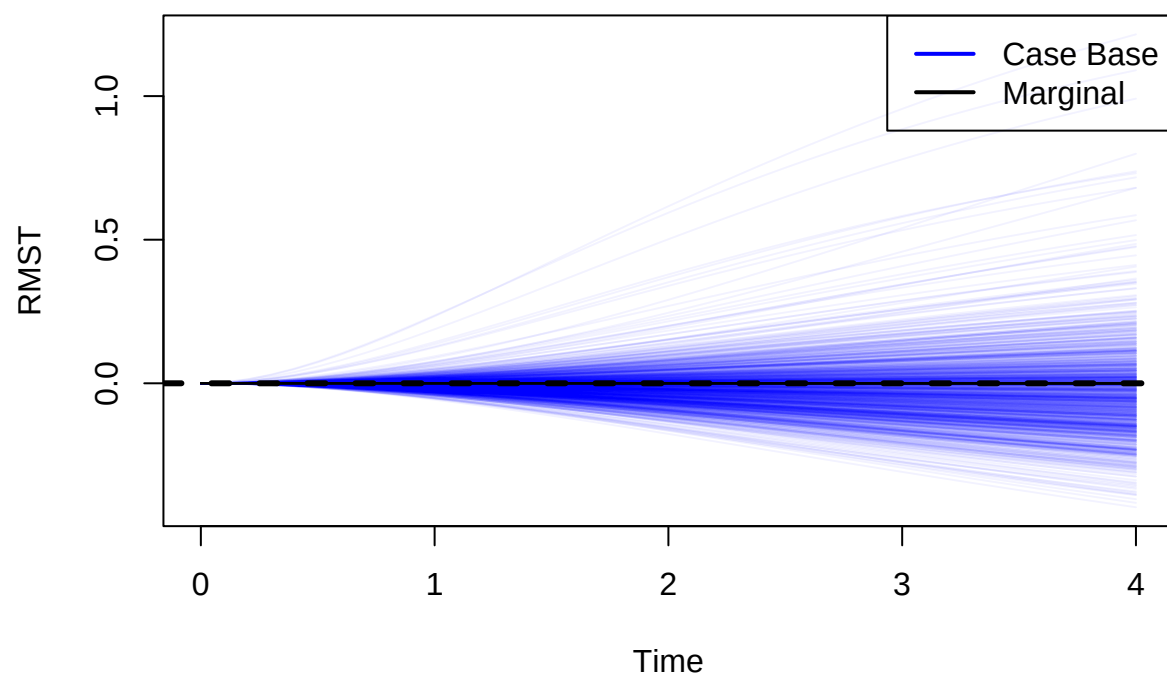
```
run.complete.ss(  
  n1, beta.A1,  
  censoring.limit.1,  
  true.estimates.null,  
  generate.plots = TRUE  
)
```



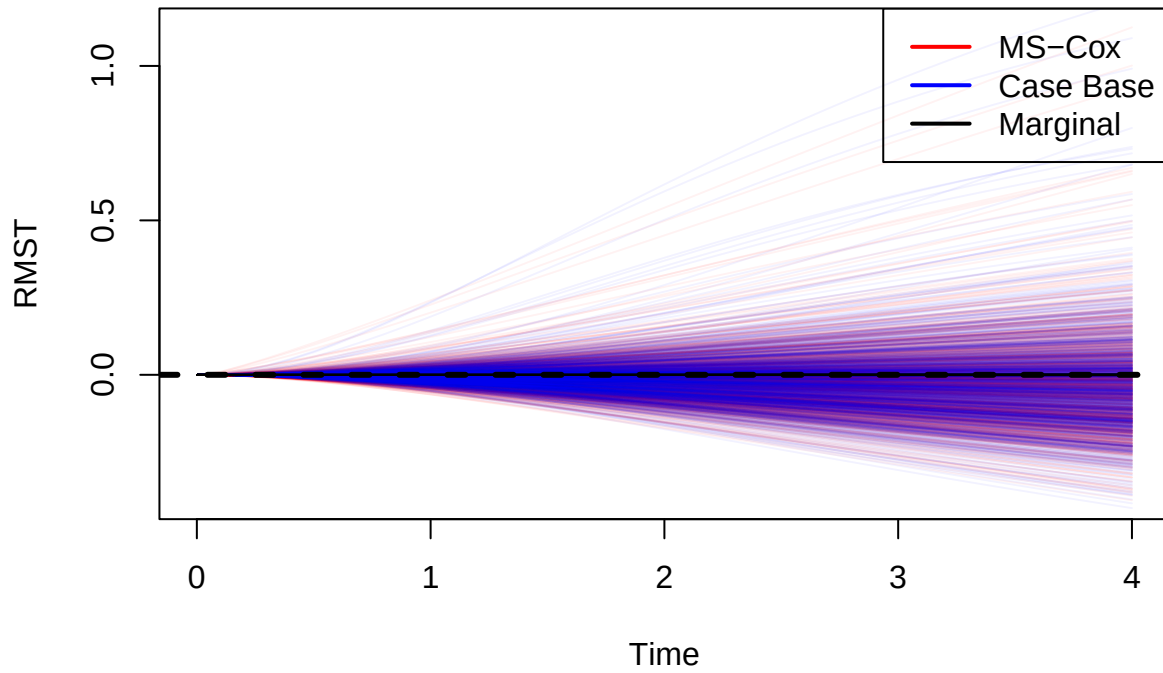
MS-Cox RMST



Case Base RMST



RMST Comparison



```
##
## =====
##
## True Mean Restricted Mean Survival Time
## =====
## 1 : 1.157282e-06
## 2 : 3.043589e-06
## 3 : 5.042438e-06
## 4 : 7.000952e-06
##
##
## =====
## MS-Cox RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.004887401
## Bias: -0.004888558 | MC-SE: 0.0008914038
## RMSE: 0.02859553
## Mean Estimated SE: 0.02739353 | Empirical SD: 0.02818866
## Coverage: 0.97 | MC-SE: 0.005394442
##
```

```

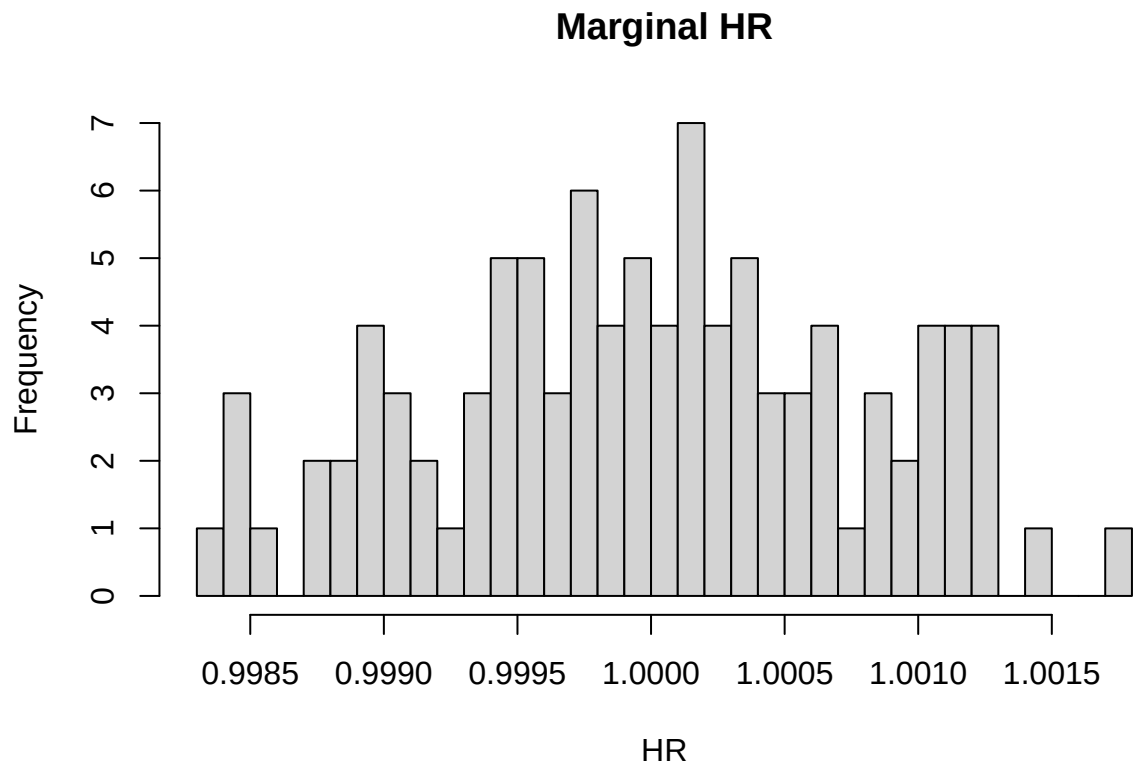
## Tau = 2
## -----
## Mean RMST: -0.01450225
## Bias: -0.0145053 | MC-SE: 0.002255555
## RMSE: 0.07275195
## Mean Estimated SE: 0.07238266 | Empirical SD: 0.07132693
## Coverage: 0.968 | MC-SE: 0.005565609
##
## Tau = 3
## -----
## Mean RMST: -0.02511931
## Bias: -0.02512435 | MC-SE: 0.003694867
## RMSE: 0.1194555
## Mean Estimated SE: 0.1202928 | Empirical SD: 0.1168419
## Coverage: 0.967 | MC-SE: 0.005648982
##
## Tau = 4
## -----
## Mean RMST: -0.03556757
## Bias: -0.03557457 | MC-SE: 0.005100574
## RMSE: 0.1650921
## Mean Estimated SE: 0.1671634 | Empirical SD: 0.1612943
## Coverage: 0.966 | MC-SE: 0.005730969
##
## =====
## Case Base RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.003672476
## Bias: -0.003673633 | MC-SE: 0.0008060566
## RMSE: 0.0257405
## Mean Estimated SE: 0.02237252 | Empirical SD: 0.02548975
## Coverage: 0.943 | MC-SE: 0.007331507
##
## Tau = 2
## -----
## Mean RMST: -0.01192917
## Bias: -0.01193222 | MC-SE: 0.002352874
## RMSE: 0.07531839
## Mean Estimated SE: 0.06740808 | Empirical SD: 0.07440442
## Coverage: 0.943 | MC-SE: 0.007331507
##
## Tau = 3
## -----
## Mean RMST: -0.02166483
## Bias: -0.02166987 | MC-SE: 0.003971813
## RMSE: 0.1273935
## Mean Estimated SE: 0.1161373 | Empirical SD: 0.1255998

```

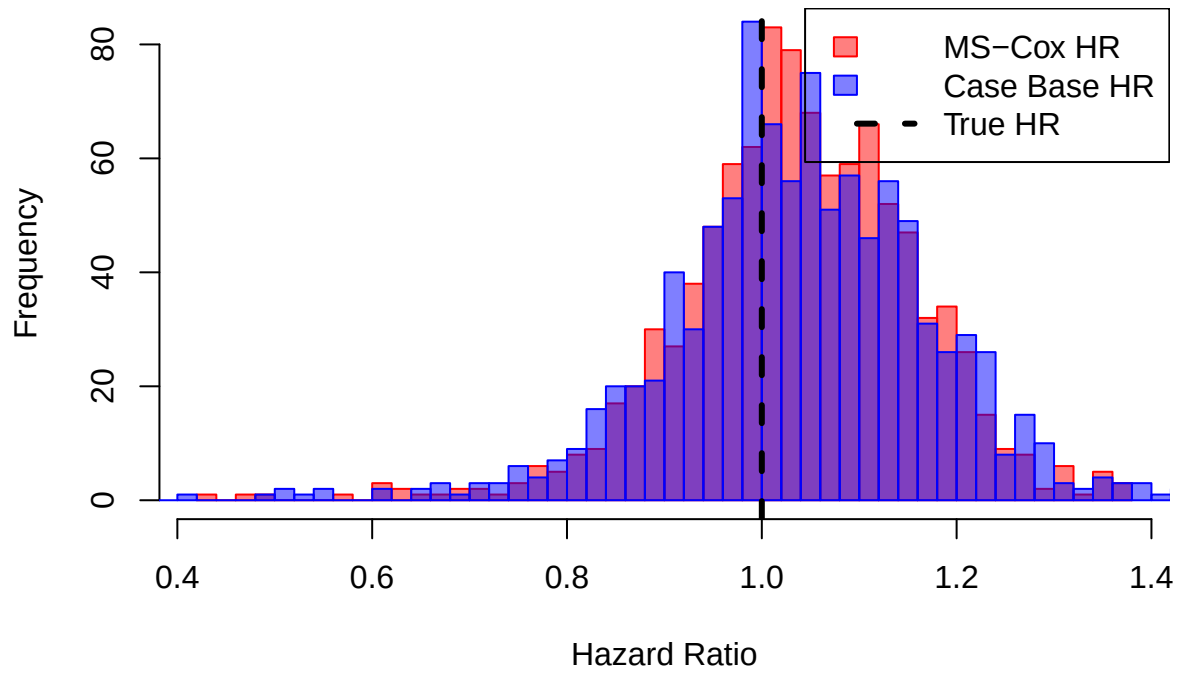
```

## Coverage: 0.943 | MC-SE: 0.007331507
##
## Tau = 4
## -----
## Mean RMST: -0.03109219
## Bias: -0.03109919 | MC-SE: 0.005418617
## RMSE: 0.1740667
## Mean Estimated SE: 0.1605116 | Empirical SD: 0.1713517
## Coverage: 0.942 | MC-SE: 0.007391617

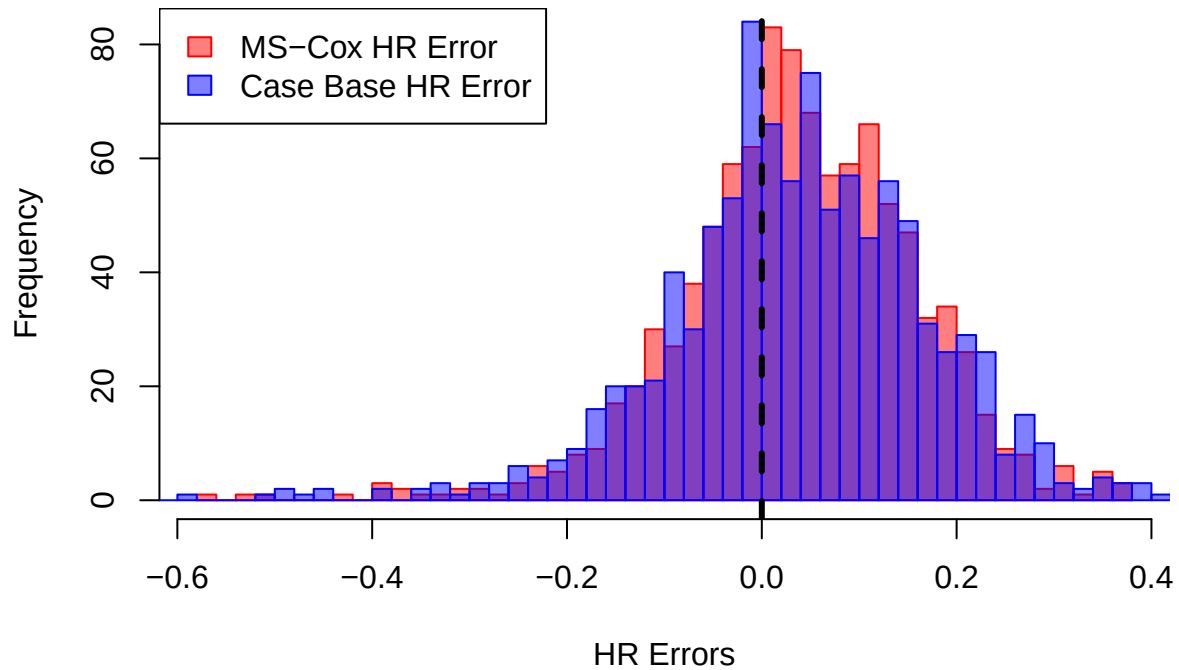
```



Hazard Ratio Comparison



HR Errors Comparison

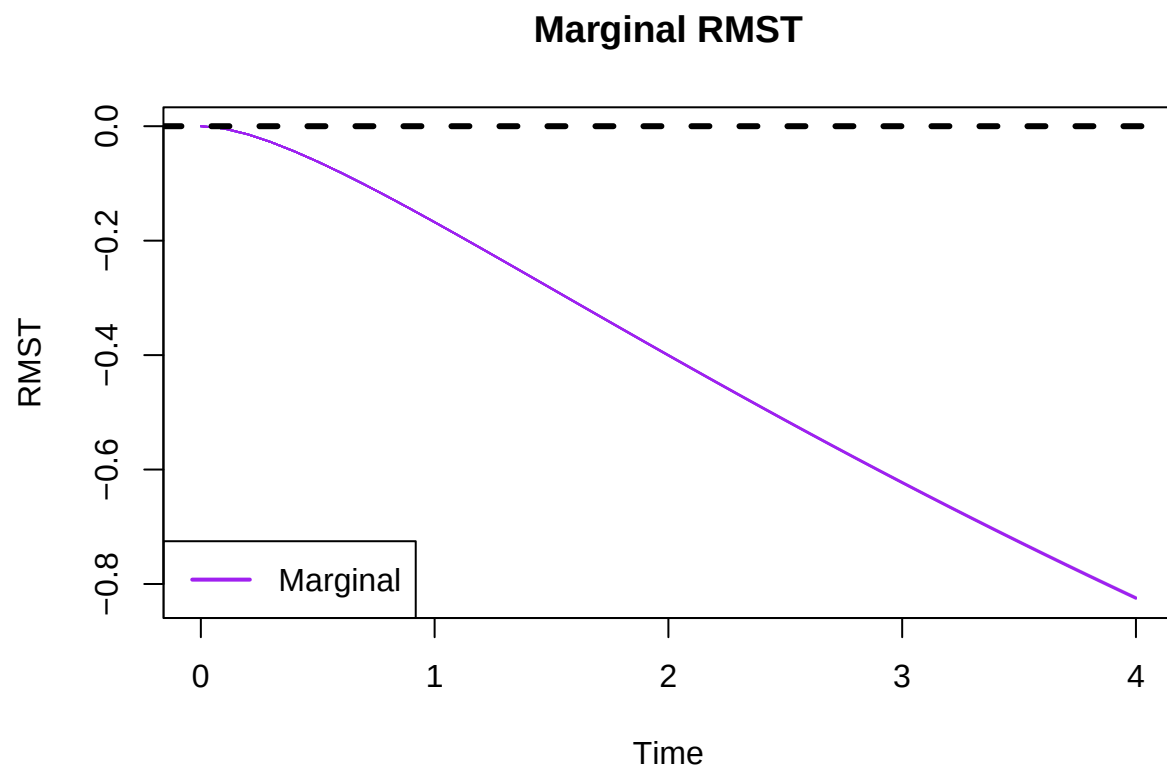


```
##
## =====
## True HR: 0.9999946
## =====
##
## =====
## MS-Cox
## =====
##
## Hazard Ratio Summary
## -----
## Mean HR: 1.029249
## Bias: 0.0292545 | MC-SE: 0.003848746
## MSE: 0.01616755
## RMSE: 0.1271517
## Mean Estimated SE: 0.1312366 | Empirical SD: 0.121708
## Type I Error/Power: 0.034
## Coverage: 0.966 | MC-SE: 0.005730969
##
##
## =====
## Case Base
## =====
##
## Hazard Ratio Summary
## -----
```

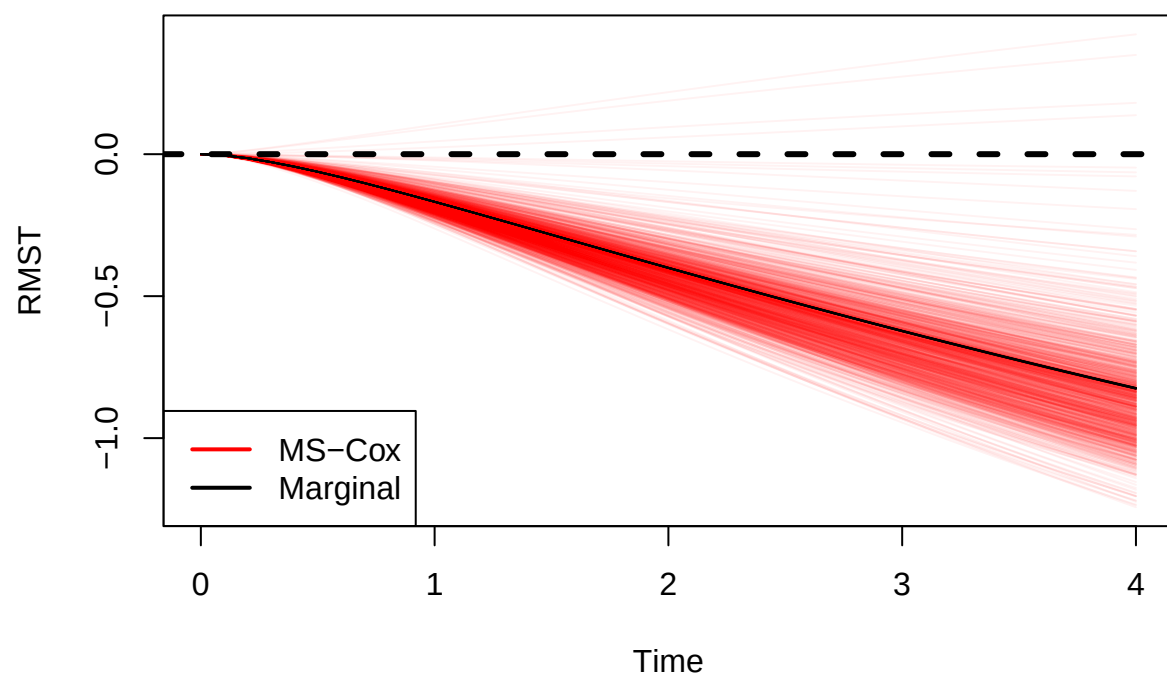
```
## Mean HR: 1.025946
## Bias: 0.02595193 | MC-SE: 0.004359047
## MSE: 0.0202971
## RMSE: 0.1424679
## Mean Estimated SE: 0.1362137 | Empirical SD: 0.1378452
## Type I Error/Power: 0.054
## Coverage: 0.946 | MC-SE: 0.007147307
```

Run Simulation Study: Setting 2

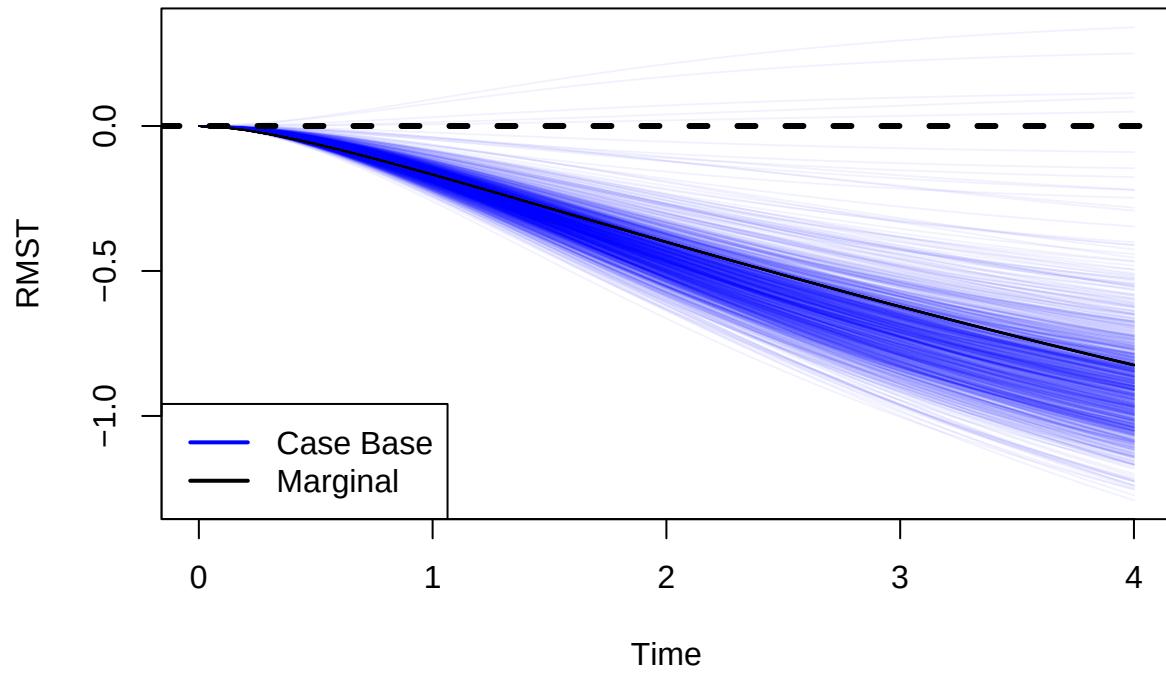
```
run.complete.ss(  
  n2, beta.A2,  
  censoring.limit.2,  
  true.estimates.strong,  
  generate.plots = TRUE  
)
```



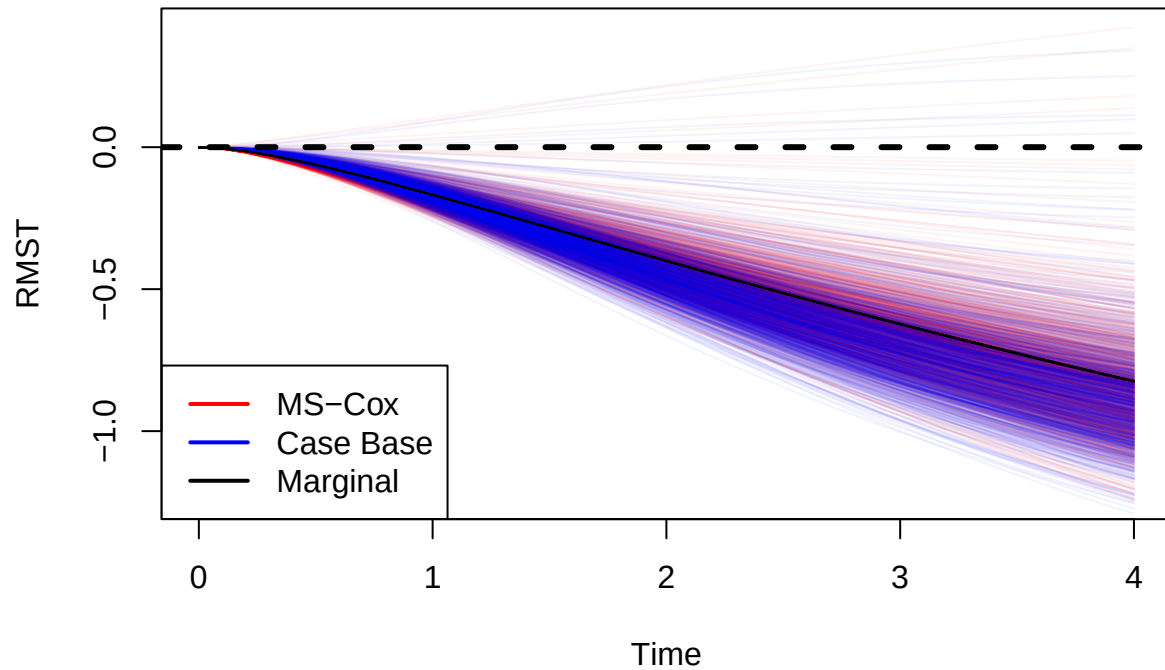
MS-Cox RMST



Case Base RMST



RMST Comparison



```
##
## =====
##
## True Mean Restricted Mean Survival Time
## =====
## 1 : -0.1676967
## 2 : -0.400371
## 3 : -0.6227171
## 4 : -0.8245784
##
##
## =====
## MS-Cox RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.1737996
## Bias: -0.006102953 | MC-SE: 0.001097309
## RMSE: 0.03521546
## Mean Estimated SE: 0.04181334 | Empirical SD: 0.03469995
## Coverage: 0.985 | MC-SE: 0.003843826
##
```

```

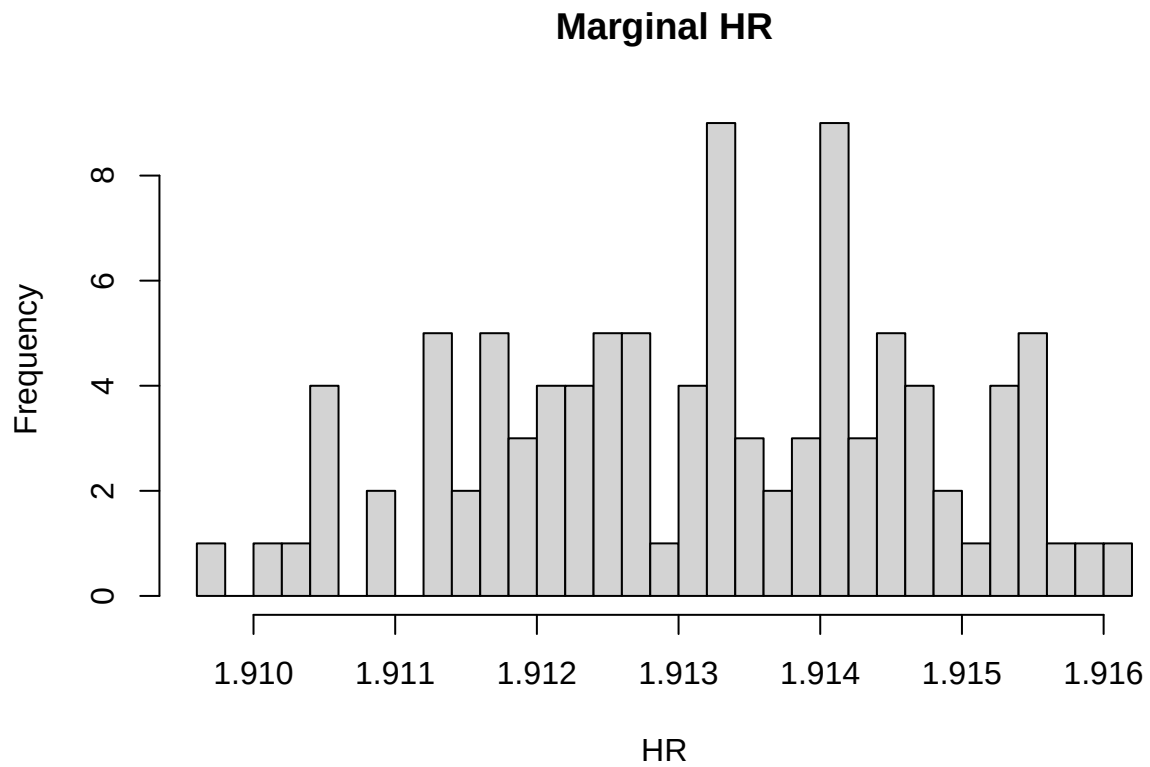
## Tau = 2
## -----
## Mean RMST: -0.4176551
## Bias: -0.01728412 | MC-SE: 0.00268541
## RMSE: 0.08661959
## Mean Estimated SE: 0.09088202 | Empirical SD: 0.08492011
## Coverage: 0.967 | MC-SE: 0.005648982
##
## Tau = 3
## -----
## Mean RMST: -0.6506802
## Bias: -0.02796312 | MC-SE: 0.0041905
## RMSE: 0.1353686
## Mean Estimated SE: 0.1316875 | Empirical SD: 0.1325153
## Coverage: 0.946 | MC-SE: 0.007147307
##
## Tau = 4
## -----
## Mean RMST: -0.8612529
## Bias: -0.03667449 | MC-SE: 0.005515439
## RMSE: 0.1781423
## Mean Estimated SE: 0.1646598 | Empirical SD: 0.1744135
## Coverage: 0.939 | MC-SE: 0.007568289
##
## =====
## Case Base RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.1662189
## Bias: 0.001477707 | MC-SE: 0.001101763
## RMSE: 0.03485473
## Mean Estimated SE: 0.02469436 | Empirical SD: 0.03484082
## Coverage: 0.879 | MC-SE: 0.01031305
##
## Tau = 2
## -----
## Mean RMST: -0.4464731
## Bias: -0.0461021 | MC-SE: 0.002834081
## RMSE: 0.1007442
## Mean Estimated SE: 0.06902601 | Empirical SD: 0.08962151
## Coverage: 0.794 | MC-SE: 0.01278921
##
## Tau = 3
## -----
## Mean RMST: -0.6983467
## Bias: -0.07562958 | MC-SE: 0.004364929
## RMSE: 0.1573321
## Mean Estimated SE: 0.1139078 | Empirical SD: 0.1380312

```

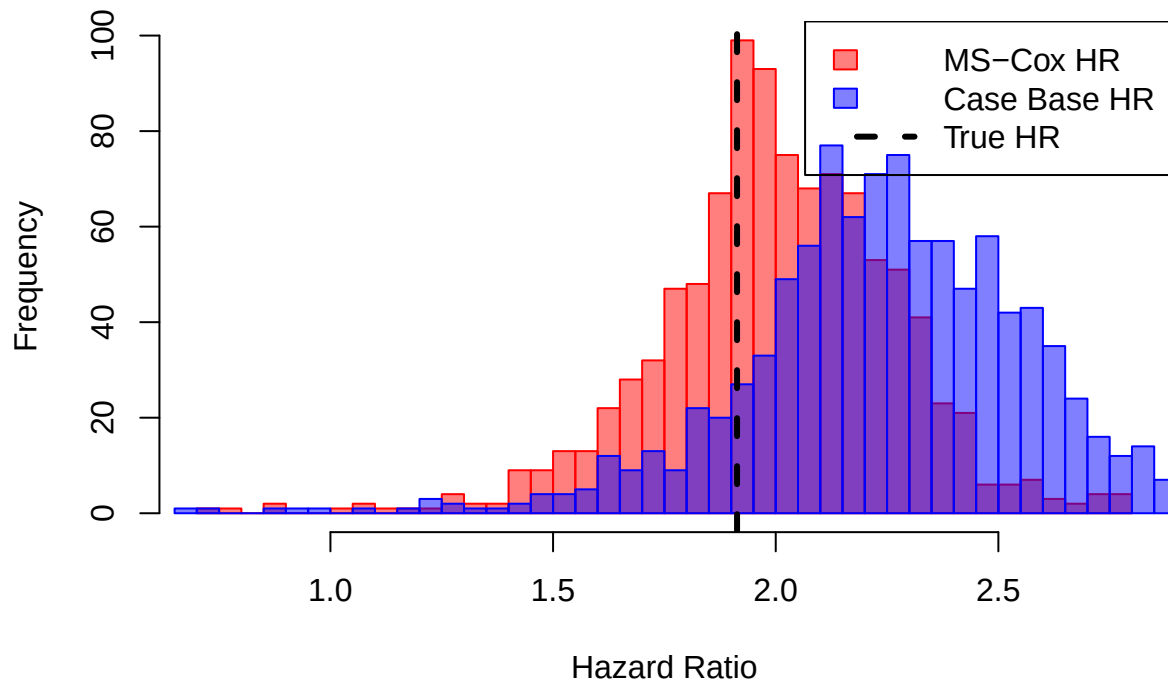
```

## Coverage: 0.815 | MC-SE: 0.01227905
##
## Tau = 4
## -----
## Mean RMST: -0.8913959
## Bias: -0.06681749 | MC-SE: 0.005584178
## RMSE: 0.1887232
## Mean Estimated SE: 0.1535955 | Empirical SD: 0.1765872
## Coverage: 0.895 | MC-SE: 0.00969407

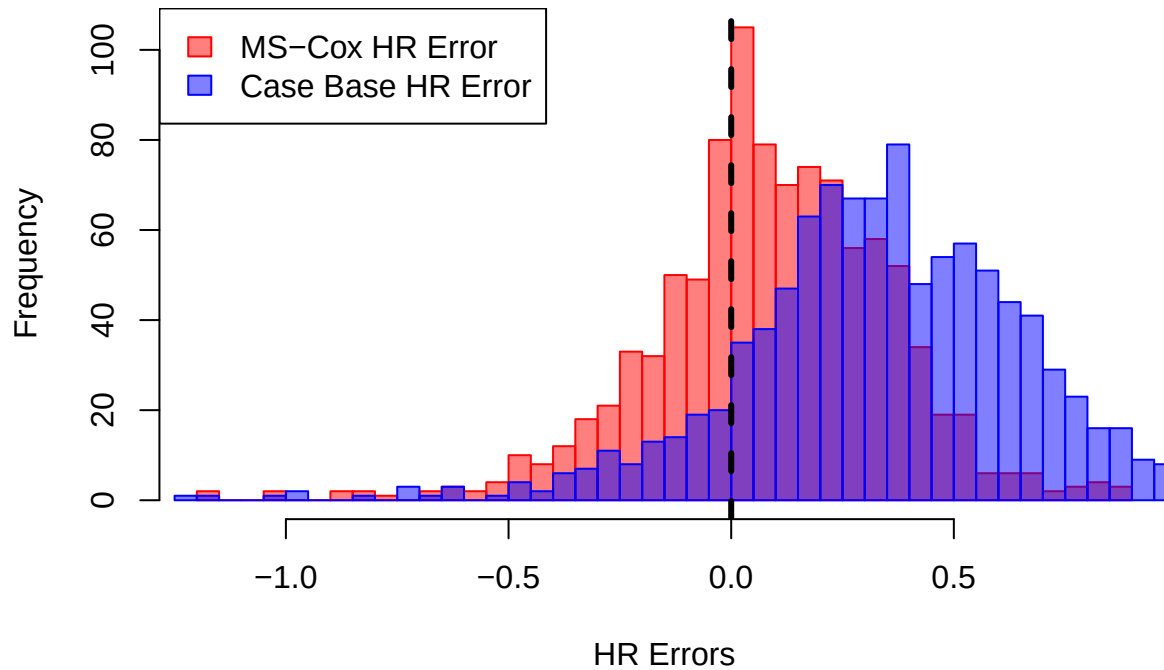
```



Hazard Ratio Comparison



HR Errors Comparison

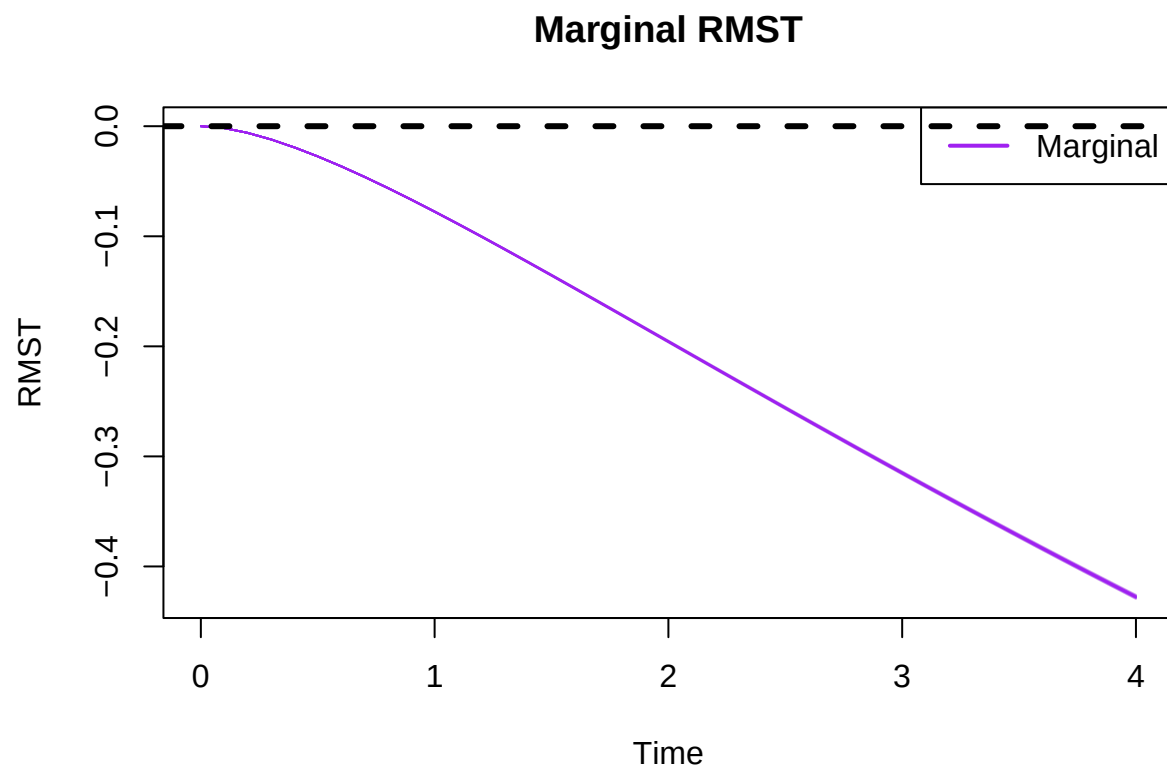


```
##
## =====
## True HR: 1.913181
## =====
##
## =====
## MS-Cox
## =====
##
## Hazard Ratio Summary
## -----
## Mean HR: 1.983243
## Bias: 0.07006206 | MC-SE: 0.008500306
## MSE: 0.08026432
## RMSE: 0.2833096
## Mean Estimated SE: 0.1417184 | Empirical SD: 0.2688033
## Type I Error/Power: 0.96
## Coverage: 0.945 | MC-SE: 0.007209369
##
##
## =====
## Case Base
## =====
##
## Hazard Ratio Summary
## -----
```

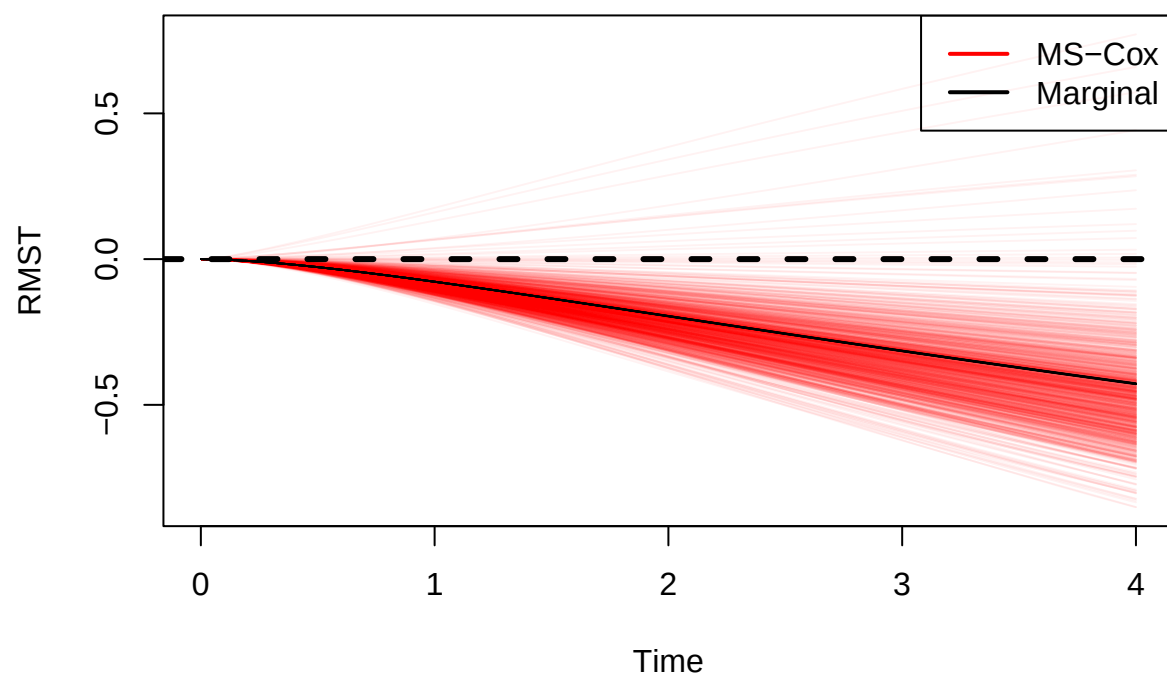
```
## Mean HR: 2.228819
## Bias: 0.3156383 | MC-SE: 0.01050939
## MSE: 0.2279821
## RMSE: 0.4774747
## Mean Estimated SE: 0.134544 | Empirical SD: 0.3323361
## Type I Error/Power: 0.978
## Coverage: 0.69 | MC-SE: 0.01462532
```

Run Simulation Study: Setting 3

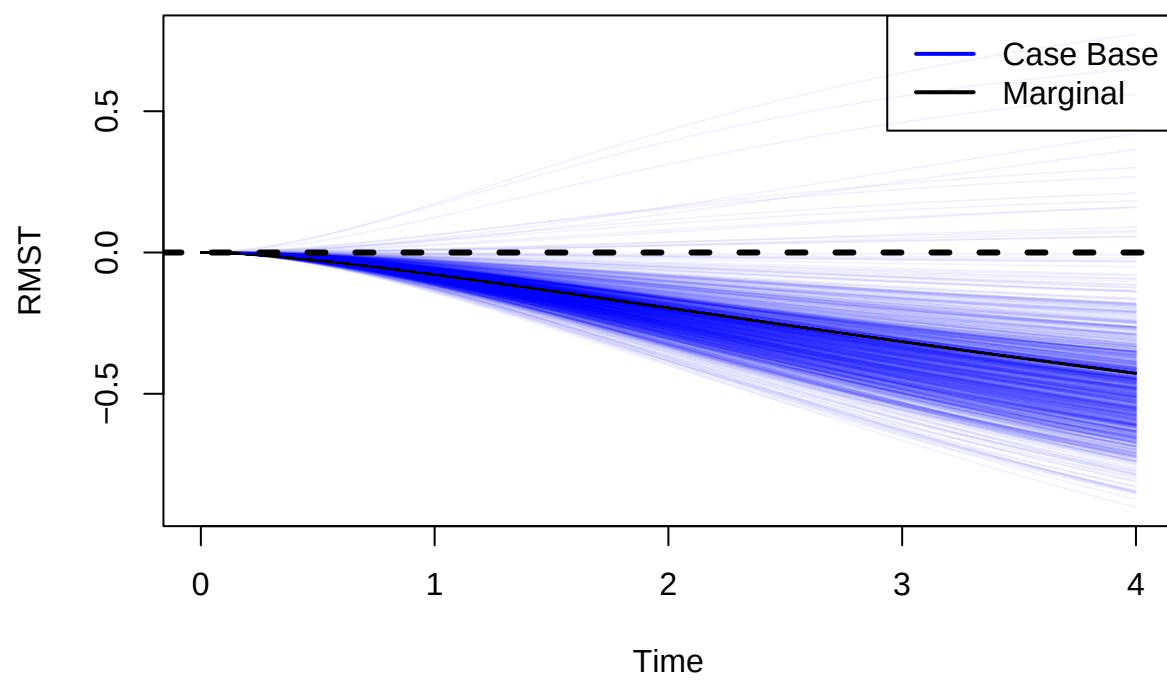
```
run.complete.ss(  
  n3, beta.A3,  
  censoring.limit.3,  
  true.estimates.moderate,  
  generate.plots = TRUE  
)
```



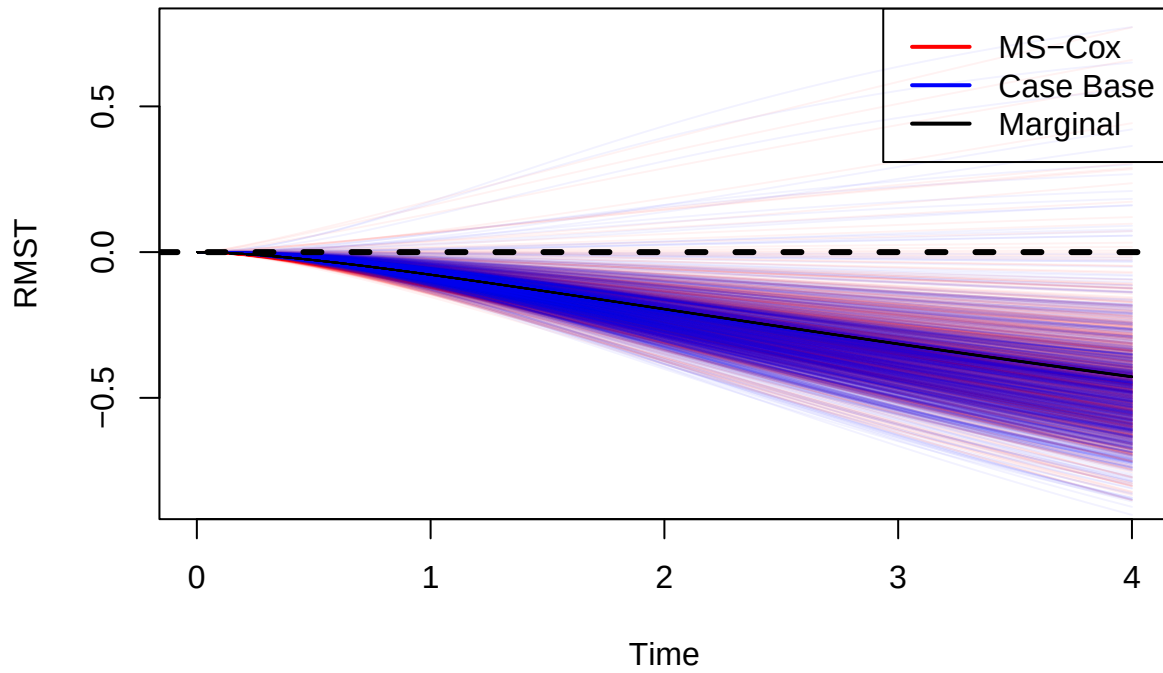
MS-Cox RMST



Case Base RMST



RMST Comparison



```
##
## =====
##
## True Mean Restricted Mean Survival Time
## =====
## 1 : -0.07760737
## 2 : -0.1956718
## 3 : -0.3150435
## 4 : -0.4276574
##
##
## =====
## MS-Cox RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.08267714
## Bias: -0.005069766 | MC-SE: 0.0009690469
## RMSE: 0.03104538
## Mean Estimated SE: 0.03463114 | Empirical SD: 0.03064395
## Coverage: 0.982 | MC-SE: 0.004204284
##
```

```

## Tau = 2
## -----
## Mean RMST: -0.2108824
## Bias: -0.01521067 | MC-SE: 0.002454862
## RMSE: 0.07906759
## Mean Estimated SE: 0.08376786 | Empirical SD: 0.07762954
## Coverage: 0.969 | MC-SE: 0.005480785
##
## Tau = 3
## -----
## Mean RMST: -0.3409267
## Bias: -0.0258832 | MC-SE: 0.003959439
## RMSE: 0.1277944
## Mean Estimated SE: 0.1306268 | Empirical SD: 0.1252085
## Coverage: 0.966 | MC-SE: 0.005730969
##
## Tau = 4
## -----
## Mean RMST: -0.4633661
## Bias: -0.03570873 | MC-SE: 0.005368086
## RMSE: 0.1733858
## Mean Estimated SE: 0.1726648 | Empirical SD: 0.1697538
## Coverage: 0.961 | MC-SE: 0.006122009
##
## =====
## Case Base RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.07468256
## Bias: 0.002924818 | MC-SE: 0.000914879
## RMSE: 0.02906409
## Mean Estimated SE: 0.02342581 | Empirical SD: 0.02893101
## Coverage: 0.931 | MC-SE: 0.008014924
##
## Tau = 2
## -----
## Mean RMST: -0.2164729
## Bias: -0.02080118 | MC-SE: 0.002574241
## RMSE: 0.08398084
## Mean Estimated SE: 0.06868738 | Empirical SD: 0.08140466
## Coverage: 0.902 | MC-SE: 0.009401915
##
## Tau = 3
## -----
## Mean RMST: -0.3593907
## Bias: -0.04434726 | MC-SE: 0.004202496
## RMSE: 0.1400357
## Mean Estimated SE: 0.1159593 | Empirical SD: 0.1328946

```

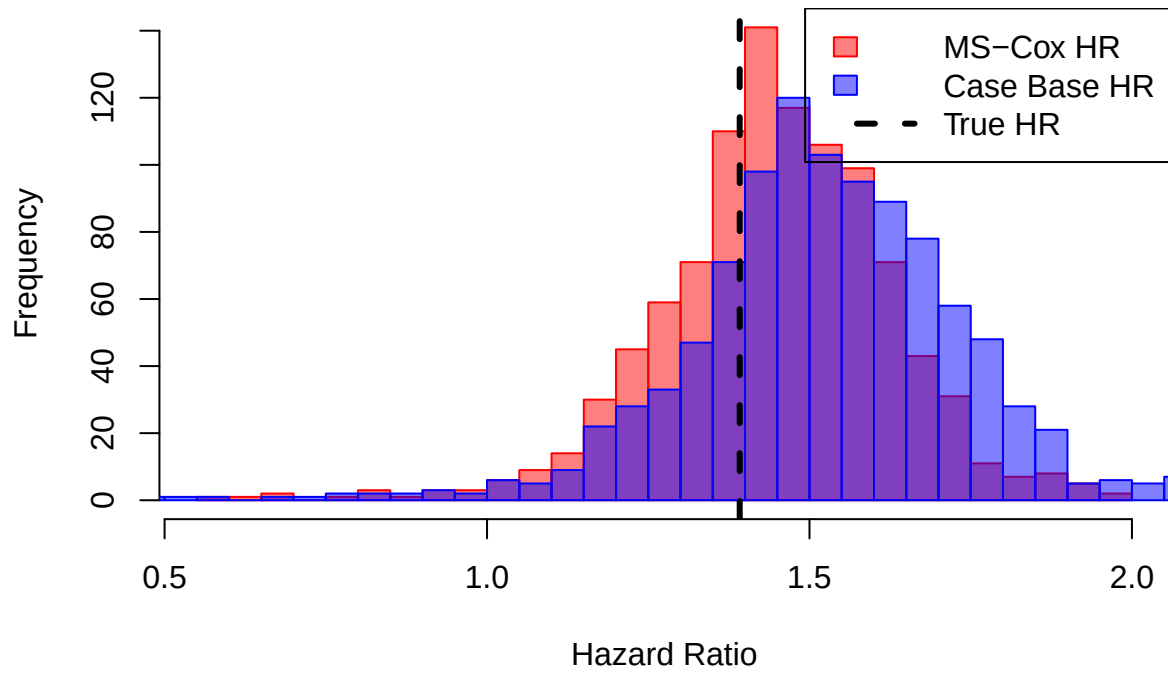
```

## Coverage: 0.9   | MC-SE: 0.009486833
##
## Tau = 4
## -----
## Mean RMST: -0.4802259
## Bias: -0.0525685   | MC-SE: 0.005571007
## RMSE: 0.1837621
## Mean Estimated SE: 0.1578835   | Empirical SD: 0.1761707
## Coverage: 0.915   | MC-SE: 0.008819014

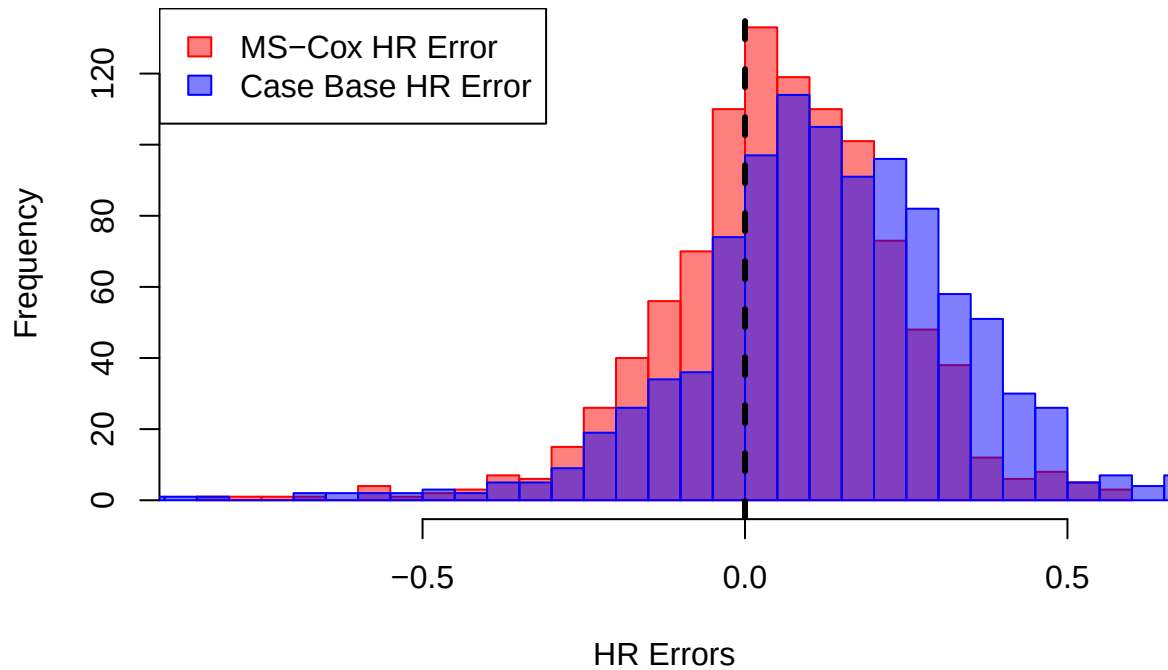
```



Hazard Ratio Comparison



HR Errors Comparison



```
##
## =====
## True HR: 1.391778
## =====
##
## =====
## MS-Cox
## =====
##
## Hazard Ratio Summary
## -----
## Mean HR: 1.435647
## Bias: 0.04386899 | MC-SE: 0.005693821
## MSE: 0.03553542
## RMSE: 0.1885084
## Mean Estimated SE: 0.1354279 | Empirical SD: 0.1800544
## Type I Error/Power: 0.789
## Coverage: 0.959 | MC-SE: 0.006270486
##
##
## =====
## Case Base
## =====
##
## Hazard Ratio Summary
## -----
```

```
## Mean HR: 1.504976
## Bias: 0.1131975 | MC-SE: 0.006662953
## MSE: 0.06112684
## RMSE: 0.2472384
## Mean Estimated SE: 0.135057 | Empirical SD: 0.2107011
## Type I Error/Power: 0.85
## Coverage: 0.869 | MC-SE: 0.01066954
```


Run Simulation Study: Setting 4

```
run.complete.ss(  
  n4, beta.A4,  
  censoring.limit.4,  
  true.estimates.moderate  
)
```

```
##  
## =====  
##  
## True Mean Restricted Mean Survival Time  
## =====  
## 1 : -0.07760737  
## 2 : -0.1956718  
## 3 : -0.3150435  
## 4 : -0.4276574  
##  
##  
## =====  
## MS-Cox RMST  
## =====  
##  
## Restricted Mean Survival Time Summary  
## -----  
##  
## Tau = 1  
## -----  
## Mean RMST: -0.08507421  
## Bias: -0.007466837 | MC-SE: 0.001219094  
## RMSE: 0.03924867  
## Mean Estimated SE: 0.04599072 | Empirical SD: 0.03855114  
## Coverage: 0.988 | MC-SE: 0.003443254  
##  
## Tau = 2  
## -----  
## Mean RMST: -0.2178636  
## Bias: -0.02219185 | MC-SE: 0.003083781  
## RMSE: 0.09996336  
## Mean Estimated SE: 0.1111314 | Empirical SD: 0.09751771  
## Coverage: 0.974 | MC-SE: 0.005032296  
##  
## Tau = 3  
## -----  
## Mean RMST: -0.3526093  
## Bias: -0.03756587 | MC-SE: 0.004973146  
## RMSE: 0.1616126  
## Mean Estimated SE: 0.1730591 | Empirical SD: 0.1572647  
## Coverage: 0.966 | MC-SE: 0.005730969  
##  
## Tau = 4  
## -----
```

```

## Mean RMST: -0.4793477
## Bias: -0.05169031 | MC-SE: 0.006745406
## RMSE: 0.2193784
## Mean Estimated SE: 0.2284912 | Empirical SD: 0.2133085
## Coverage: 0.955 | MC-SE: 0.006555532
##
##
## =====
## Case Base RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.07677088
## Bias: 0.0008364894 | MC-SE: 0.001177639
## RMSE: 0.03723098
## Mean Estimated SE: 0.03191703 | Empirical SD: 0.03724021
## Coverage: 0.933 | MC-SE: 0.00790639
##
## Tau = 2
## -----
## Mean RMST: -0.2222491
## Bias: -0.02657733 | MC-SE: 0.003301822
## RMSE: 0.1076916
## Mean Estimated SE: 0.09294632 | Empirical SD: 0.1044128
## Coverage: 0.914 | MC-SE: 0.00886589
##
## Tau = 3
## -----
## Mean RMST: -0.3688103
## Bias: -0.05376684 | MC-SE: 0.005390803
## RMSE: 0.1786689
## Mean Estimated SE: 0.156323 | Empirical SD: 0.1704722
## Coverage: 0.91 | MC-SE: 0.009049862
##
## Tau = 4
## -----
## Mean RMST: -0.4928804
## Bias: -0.06522303 | MC-SE: 0.007164089
## RMSE: 0.2356414
## Mean Estimated SE: 0.2124701 | Empirical SD: 0.2265484
## Coverage: 0.927 | MC-SE: 0.008226239
##
##
## =====
## True HR: 1.391778
## =====
##
## =====
## MS-Cox
## =====

```

```

##
## Hazard Ratio Summary
## -----
## Mean HR: 1.456399
## Bias: 0.06462114 | MC-SE: 0.007377941
## MSE: 0.06153404
## RMSE: 0.2480606
## Mean Estimated SE: 0.1802018 | Empirical SD: 0.233311
## Type I Error/Power: 0.639
## Coverage: 0.956 | MC-SE: 0.006485677
##
##
## =====
## Case Base
## =====
##
## Hazard Ratio Summary
## -----
## Mean HR: 1.522871
## Bias: 0.1310925 | MC-SE: 0.008807012
## MSE: 0.1027226
## RMSE: 0.3205036
## Mean Estimated SE: 0.1829533 | Empirical SD: 0.2785022
## Type I Error/Power: 0.711
## Coverage: 0.903 | MC-SE: 0.009359006

```

Run Simulation Study: Setting 5

```
run.complete.ss(  
  n5, beta.A5,  
  censoring.limit.5,  
  true.estimates.moderate  
)
```

```
##  
## =====  
##  
## True Mean Restricted Mean Survival Time  
## =====  
## 1 : -0.07760737  
## 2 : -0.1956718  
## 3 : -0.3150435  
## 4 : -0.4276574  
##  
##  
## =====  
## MS-Cox RMST  
## =====  
##  
## Restricted Mean Survival Time Summary  
## -----  
##  
## Tau = 1  
## -----  
## Mean RMST: -0.09024178  
## Bias: -0.01263441 | MC-SE: 0.00161456  
## RMSE: 0.0525721  
## Mean Estimated SE: 0.06070955 | Empirical SD: 0.05105687  
## Coverage: 0.981 | MC-SE: 0.004317291  
##  
## Tau = 2  
## -----  
## Mean RMST: -0.2324634  
## Bias: -0.0367916 | MC-SE: 0.004121714  
## RMSE: 0.1353704  
## Mean Estimated SE: 0.1466551 | Empirical SD: 0.13034  
## Coverage: 0.97 | MC-SE: 0.005394442  
##  
## Tau = 3  
## -----  
## Mean RMST: -0.3768026  
## Bias: -0.06175909 | MC-SE: 0.006669824  
## RMSE: 0.219673  
## Mean Estimated SE: 0.2277717 | Empirical SD: 0.2109183  
## Coverage: 0.963 | MC-SE: 0.005969171  
##  
## Tau = 4  
## -----
```

```

## Mean RMST: -0.5120562
## Bias: -0.08439885 | MC-SE: 0.009045422
## RMSE: 0.2980956
## Mean Estimated SE: 0.300006 | Empirical SD: 0.2860414
## Coverage: 0.946 | MC-SE: 0.007147307
##
##
## =====
## Case Base RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.08120692
## Bias: -0.003599548 | MC-SE: 0.001594436
## RMSE: 0.05052367
## Mean Estimated SE: 0.04329993 | Empirical SD: 0.0504205
## Coverage: 0.923 | MC-SE: 0.008430362
##
## Tau = 2
## -----
## Mean RMST: -0.2345542
## Bias: -0.03888245 | MC-SE: 0.004435665
## RMSE: 0.1454898
## Mean Estimated SE: 0.1256959 | Empirical SD: 0.140268
## Coverage: 0.904 | MC-SE: 0.009315793
##
## Tau = 3
## -----
## Mean RMST: -0.3889512
## Bias: -0.07390769 | MC-SE: 0.007236111
## RMSE: 0.2403566
## Mean Estimated SE: 0.2113496 | Empirical SD: 0.2288259
## Coverage: 0.906 | MC-SE: 0.009228434
##
## Tau = 4
## -----
## Mean RMST: -0.5200658
## Bias: -0.09240845 | MC-SE: 0.009637085
## RMSE: 0.3183078
## Mean Estimated SE: 0.2876424 | Empirical SD: 0.3047514
## Coverage: 0.913 | MC-SE: 0.008912407
##
##
## =====
## True HR: 1.391778
## =====
##
## =====
## MS-Cox
## =====

```

```

##
## Hazard Ratio Summary
## -----
## Mean HR: 1.501318
## Bias: 0.1095401 | MC-SE: 0.01128092
## MSE: 0.1495646
## RMSE: 0.3867359
## Mean Estimated SE: 0.2392413 | Empirical SD: 0.356734
## Type I Error/Power: 0.441
## Coverage: 0.948 | MC-SE: 0.007021111
##
##
## =====
## Case Base
## =====
##
## Hazard Ratio Summary
## -----
## Mean HR: 1.56233
## Bias: 0.170552 | MC-SE: 0.01322492
## MSE: 0.2254961
## RMSE: 0.4748643
## Mean Estimated SE: 0.2493552 | Empirical SD: 0.4182087
## Type I Error/Power: 0.495
## Coverage: 0.9 | MC-SE: 0.009486833

```

Run Simulation Study: Setting 6

```
run.complete.ss(  
  n6, beta.A6,  
  censoring.limit.6,  
  true.estimates.moderate  
)
```

```
##  
## =====  
##  
## True Mean Restricted Mean Survival Time  
## =====  
## 1 : -0.07760737  
## 2 : -0.1956718  
## 3 : -0.3150435  
## 4 : -0.4276574  
##  
##  
## =====  
## MS-Cox RMST  
## =====  
##  
## Restricted Mean Survival Time Summary  
## -----  
##  
## Tau = 1  
## -----  
## Mean RMST: -0.08708602  
## Bias: -0.009478645 | MC-SE: 0.001082281  
## RMSE: 0.03549656  
## Mean Estimated SE: 0.03860898 | Empirical SD: 0.03422473  
## Coverage: 0.985 | MC-SE: 0.003843826  
##  
## Tau = 2  
## -----  
## Mean RMST: -0.2225206  
## Bias: -0.02684885 | MC-SE: 0.002744671  
## RMSE: 0.09081049  
## Mean Estimated SE: 0.09321134 | Empirical SD: 0.08679411  
## Coverage: 0.968 | MC-SE: 0.005565609  
##  
## Tau = 3  
## -----  
## Mean RMST: -0.3599376  
## Bias: -0.04489409 | MC-SE: 0.004432866  
## RMSE: 0.1471262  
## Mean Estimated SE: 0.1450531 | Empirical SD: 0.1401795  
## Coverage: 0.949 | MC-SE: 0.006956939  
##  
## Tau = 4  
## -----
```

```

## Mean RMST: -0.4938451
## Bias: -0.06618775 | MC-SE: 0.006083219
## RMSE: 0.2033454
## Mean Estimated SE: 0.1941676 | Empirical SD: 0.1923683
## Coverage: 0.938 | MC-SE: 0.007626008
##
##
## =====
## Case Base RMST
## =====
##
## Restricted Mean Survival Time Summary
## -----
##
## Tau = 1
## -----
## Mean RMST: -0.08115419
## Bias: -0.003546819 | MC-SE: 0.001062881
## RMSE: 0.03378117
## Mean Estimated SE: 0.02895153 | Empirical SD: 0.03361126
## Coverage: 0.937 | MC-SE: 0.007683163
##
## Tau = 2
## -----
## Mean RMST: -0.2275889
## Bias: -0.03191718 | MC-SE: 0.002881042
## RMSE: 0.09649254
## Mean Estimated SE: 0.08220987 | Empirical SD: 0.09110655
## Coverage: 0.9 | MC-SE: 0.009486833
##
## Tau = 3
## -----
## Mean RMST: -0.3671774
## Bias: -0.05213392 | MC-SE: 0.004582498
## RMSE: 0.1539358
## Mean Estimated SE: 0.1352033 | Empirical SD: 0.1449113
## Coverage: 0.91 | MC-SE: 0.009049862
##
## Tau = 4
## -----
## Mean RMST: -0.4788787
## Bias: -0.0512213 | MC-SE: 0.005959359
## RMSE: 0.1951975
## Mean Estimated SE: 0.1801003 | Empirical SD: 0.1884515
## Coverage: 0.931 | MC-SE: 0.008014924
##
##
## =====
## True HR: 1.391778
## =====
##
## =====
## MS-Cox
## =====

```



```

##
## Hazard Ratio Summary
## -----
## Mean HR: 1.466587
## Bias: 0.07480867 | MC-SE: 0.006540331
## MSE: 0.05093354
## RMSE: 0.2256846
## Mean Estimated SE: 0.1505033 | Empirical SD: 0.2068234
## Type I Error/Power: 0.761
## Coverage: 0.936 | MC-SE: 0.007739767
##
##
## =====
## Case Base
## =====
##
## Hazard Ratio Summary
## -----
## Mean HR: 1.513519
## Bias: 0.1217408 | MC-SE: 0.007277966
## MSE: 0.07285053
## RMSE: 0.2699084
## Mean Estimated SE: 0.1555925 | Empirical SD: 0.2301495
## Type I Error/Power: 0.791
## Coverage: 0.89 | MC-SE: 0.009894443

```